

**FORGEGUARD: AN AI-POWERED DEEPPFAKE AND
IMAGE MANIPULATION DETECTION SYSTEM**

A project report submitted in partial fulfillment of the
requirements for the award of degree of

Bachelor of Technology
In
Computer Science and Engineering
“B.Tech (CSE)”

Under the Supervision of
Pardeep Singh (Dean Academics & HOD(CSE))
Department of Computer Science and Engineering

Submitted by :

Name of the Student : Pushkar Anand

Enrollment No : 00223802722

Mentor's Name

Mrs. Sarita Yadav



GURU TEGH 4TH CENTENARY ENGINEERING COLLEGE,
G-8 AREA, RAJOURI GARDEN, NEW DELHI-110064
(Affiliated to Guru Gobind Singh Indraprastha University)

(BATCH- 2022-26)

DECLARATION

I hereby certify that the work carried out in this project is an authentic record of my own. I hereby submit this project, as approved by my supervisor/ project guide. I have read and understood the rules related to the project, inventions and innovations and agree to be bound by them. I also declare that this work has not been submitted earlier for the award of any degree or diploma to the best of my knowledge and belief.

Verified by :-

Pardeep Singh

HOD (CSE Department)

Pushkar Anand

Enrollment No: 00223802722

GTB4CEC, G-8 Area, Rajouri Garden,

New Delhi - 110064

Date :-

CERTIFICATE OF THE MENTOR

This is to certify that, the project titled “**ForgeGuard: An AI-Powered Deepfake and Image Manipulation Detection System**” submitted by “**Pushkar Anand**” for the award of 4 years Bachelor’s Degree in Computer Science and Engineering is a record of authentic work carried out by him under my Supervision. The matter embodied in this project is the original work of the candidate and has not been submitted for the award of any other degree/diploma in any other university. It is further certified that he/she has worked with me and under the guidance/supervision of our Head of Department & Dean Academics- S. Pardeep Singh as per the prescribed syllabus,norms and the regulations of GGSIPU.

(Mrs. Sarita Yadav)

Assistant Professor

Department of Computer Science & Engineering

GTB4CEC, Rajouri Garden, New Delhi - 110064

ACKNOWLEDGMENT

I would like to thank all my respected teachers (Mrs. Sarita Yadav, Pardeep Singh and other faculty members of the Computer Department in chronological order) for encouraging me and giving me an opportunity to work in the Industry/Major Project on the project “ForgeGuard: An AI-Powered Deepfake and Image Manipulation Detection System”. This project “ForgeGuard” gave me an opportunity and pleasure to explore new and challenging field and let me achieve maximum out of my project.

I also wish my gratitude to our Director ,GTB4CEC “**Gurmeet Singh Bali** ” and TPO (Training and placement officer) who helped me at every point and guided me with the right direction to step forward in my career to grab all opportunities during the course of my study in GTB4CEC from 2022-26 and to undergo this training/Major Project.

“I would also like to extend my heartfelt thanks to my family and friends whose constant moral support and encouragement kept me motivated throughout the course of this project. Their belief in my abilities was a source of strength during the challenging phases of this work.”

Signature of the student

Name of the Student

(Roll No)

Signature

Head of Department

Computer Sc. & Engg

TABLE OF CONTENTS

Declaration	iii
Certificate from Mentor	iv
Acknowledgement	v
Table of Contents	vi
List of Figures	viii
List of Tables	ix
Abstract / Summary	x
Chapter 1: Introduction to the Organisation	1
1.1 About the Institution	2
1.2 Vision and Mission	3
1.3 Organisational Structure	5
1.4 Departments and Facilities	7
1.5 The Department of Computer Science and Engineering	9
1.6 Role of the Student	12
1.7 Summary	14
Chapter 2: Introduction to the Project	16
2.1 Problem Statement	17
2.2 Motivation	19
2.3 Objectives	21
2.4 Project Scope	22
2.5 Limitations	24
2.6 Technology Stack Overview	25
2.7 Project Workflow	28
2.8 Summary	30
Chapter 3: Literature Review and Survey	32
3.1 Introduction to the Literature Review	33
3.2 Review of Research Papers	34
3.3 Comparative Analysis of Existing Systems	48
3.4 Gap Analysis	52
3.5 Summary	54
Chapter 4: System Analysis and Requirements	56
4.1 Functional Requirements	57
4.2 Non-Functional Requirements	60
4.3 Hardware and Software Specifications	63
4.4 Feasibility Study	66
4.5 Summary	70

Chapter 5: System Design	72
5.1 Overall System Architecture	73
5.2 Data Flow Diagrams	76
5.3 UML Diagrams	82
5.4 Database Design	91
5.5 API Design	96
5.6 Frontend Design	101
5.7 Summary	105

TABLE OF CONTENTS (CONTINUED)

Chapter 6: Implementation Details	107
6.1 Project Structure and Module Overview	108
6.2 Image Manipulation Detection Pipeline (IMDModel)	112
6.3 Error Level Analysis (ELA)	120
6.4 Video Deepfake Detection Pipeline	126
6.5 Face Detection Engine	134
6.6 REST API Layer	140
6.7 Frontend Implementation	148
6.8 Database Integration	155
6.9 Exception Handling Framework	158
6.10 Summary	162
Chapter 7: Testing and Results	164
7.1 Testing Strategy	165
7.2 Unit Tests	167
7.3 Functional Test Cases	169
7.4 Performance and Accuracy Results	178
7.5 System Screenshots	183
7.6 Summary	188
Chapter 8: Conclusion and Future Scope	190
8.1 Summary of Work Done	191
8.2 Achievements	193
8.3 Limitations	195
8.4 Future Enhancements	197
8.5 Conclusion	200
Chapter 9: Deployment and Security Analysis	202
9.1 Deployment Architecture	203
9.2 Security Considerations	207
9.3 User Manual	212
9.4 Summary	217
Bibliography / References	219

LIST OF FIGURES

Figure No.	Figure Description	Page No.
Figure 1.1	GTB4CEC College Campus / Logo	2
Figure 1.2	Organisational Chart of GTB4CEC	6
Figure 1.3	Department of CSE – Lab and Infrastructure	10
Figure 2.1	Deepfake Examples – Real vs. Generated Faces	18
Figure 2.2	Project Workflow Flowchart	29
Figure 2.3	Technology Stack Diagram	26
Figure 3.1	Research Trend – Publications on Deepfake Detection (2017–2024)	35
Figure 3.2	Comparative Chart – Existing Deepfake Detection Tools	50
Figure 4.1	Feasibility Analysis – Pie Chart	68
Figure 4.2	Requirements Summary Table Diagram	62
Figure 5.1	ForgeGuard High-Level System Architecture	74
Figure 5.2	Level 0 – Context DFD	77
Figure 5.3	Level 1 – System DFD (Image Module)	79
Figure 5.4	Level 2 – DFD (Image Pipeline Details)	81
Figure 5.5	Use Case Diagram – ForgeGuard	83
Figure 5.6	Class Diagram – Core Pipeline Classes	86
Figure 5.7	Sequence Diagram – Image Detection Flow	89
Figure 5.8	Entity-Relationship (ER) Diagram	92
Figure 5.9	API Response Structure	98
Figure 6.1	Project Directory Structure	109
Figure 6.2	IMDModel Architecture – CNN Layers	114
Figure 6.3	ELA Process – Original vs. Re-compressed vs. Difference	121
Figure 6.4	EfficientNet B7 Scaling Architecture	128
Figure 6.5	MTCNN Face Detection Pipeline	135
Figure 6.6	ForgeGuard Home Page – Screenshot	149
Figure 6.7	Image Upload Page – Screenshot	150
Figure 6.8	Image Result Page – AUTHENTIC Result	151
Figure 6.9	Image Result Page – TAMPERED Result	152
Figure 6.10	Video Upload Page – Screenshot	153
Figure 6.11	API Test via Postman – Detect Image	154

Figure 6.12	API Test via Postman – Detect Video	155
Figure 7.1	Test Case Execution Summary	170
Figure 7.2	Accuracy Comparison – Bar Chart	180
Figure 7.3	Processing Time vs. File Size – Graph	182
Figure 7.4	Confusion Matrix – Image Detection	179
Figure 7.5	Screenshot – No Face Detected Response	185
Figure 9.1	Deployment Architecture Diagram	204
Figure 9.2	Request-Response Security Flow	209

LIST OF TABLES

Table No.	Table Description	Page No.
Table 3.1	Comparative Analysis of Existing Deepfake Detection Systems	49
Table 3.2	Summary of Research Papers Reviewed	46
Table 4.1	Functional Requirements Table	58
Table 4.2	Non-Functional Requirements Table	61
Table 4.3	Hardware Specifications	64
Table 4.4	Software Specifications	65
Table 4.5	Feasibility Analysis Summary	69
Table 5.1	API Endpoints Summary	97
Table 5.2	Database Tables – Field Descriptions	93
Table 6.1	Module Descriptions	111
Table 6.2	Technologies and Libraries Used	112
Table 6.3	IMDModel Layer Architecture Summary	115
Table 6.4	API Request and Response Fields – Image Endpoint	141
Table 6.5	API Request and Response Fields – Video Endpoint	143
Table 7.1	Positive Test Cases – Image Detection	171
Table 7.2	Negative Test Cases – Image Detection	173
Table 7.3	Positive Test Cases – Video Detection	175
Table 7.4	Negative Test Cases – Video Detection	177
Table 7.5	Performance Results – Accuracy and Confidence	179
Table 7.6	Comparison: ForgeGuard vs. Existing Tools	181
Table 9.1	Security Threats and Mitigations	210

ABSTRACT / SUMMARY

The rapid advancement of Generative Adversarial Networks (GANs) and deep learning-based image synthesis techniques has given rise to a category of highly realistic synthetic media, commonly known as "deepfakes." These automated forgery tools can manipulate human facial identities in videos and alter pixel-level content in images with imperceptible accuracy, posing unprecedented threats to digital trust, media authenticity, political discourse, legal evidence, and personal privacy. The ForgeGuard project addresses this critical and growing threat through the design and implementation of a comprehensive, multi-modal, AI-powered forgery detection system capable of analyzing both still images and video media.

ForgeGuard employs a dual-layer detection architecture. For image analysis, the system implements a two-level pipeline: **Level-1** performs EXIF metadata forensics by inspecting the software signatures embedded in image headers to identify known editing tools such as Adobe Photoshop or GIMP. **Level-2** applies Error Level Analysis (ELA), a well-established image forensics technique, which re-compresses the image at a reduced JPEG quality factor and amplifies pixel-level differences to generate a forensic ELA image. This ELA image is then fed into a custom Convolutional Neural Network — the **IMDModel** — trained on the CASIA Tampered Image Detection benchmark dataset, which classifies the input as either AUTHENTIC or TAMPERED with a probability distribution across both classes.

For video analysis, ForgeGuard leverages the **EfficientNet B7** backbone architecture, pre-trained and fine-tuned on deepfake video datasets spanning over 100,000 clips. The video pipeline uses MTCNN (Multi-Task Cascaded Convolutional Networks) with training-matched detection thresholds of [0.7, 0.8, 0.8] to extract face regions from uniformly sampled frames. These face crops undergo isotropic resizing to 380×380 pixels, are normalized with ImageNet statistics, and are classified using the EfficientNet B7 model. A novel *confident strategy* aggregation function then fuses per-frame predictions into a single verdict of REAL or FAKE.

The entire system is deployed as a production-grade Django 5.0 REST API with a responsive HTML/CSS/JavaScript frontend. The API exposes dedicated endpoints for image detection, video detection, and a unified auto-detecting endpoint. The system achieves over 89% confidence on out-of-distribution deepfake video samples and demonstrates robust handling of edge cases including face-less videos, corrupt media, and unsupported file formats. A thread-safe Singleton model caching architecture prevents redundant model loading and eliminates memory overflow on concurrent requests.

[TRANSPARENCY SHEET]

CHAPTER 1

CHAPTER 1

INTRODUCTION

1.1 Background of Digital Forgery and Deepfakes

1.1.1 Historical Context: Pre-Digital vs. Digital Manipulation

The human endeavor to manipulate visual records is nearly as old as the medium of photography itself. In the mid-19th century, photographers quickly discovered that by dodging and burning during the exposure of a print, or by physically combining multiple negatives, they could alter the apparent reality of an image. One of the earliest known examples dates back to the 1860s, involving the composite portrait of U.S. President Abraham Lincoln, whose head was superimposed onto the body of Southern politician John Calhoun. Throughout the 20th century, state actors frequently utilized airbrushing and darkroom compositing to remove political dissidents from official photographs, a practice heavily documented in Stalinist Russia and Maoist China. However, these analog techniques were painstakingly slow, required immense specialized skill, and often left physical traces on the negatives or prints that could be detected by expert examiners.

The dawn of the digital age in the late 20th century fundamentally transformed this landscape. The transition from chemical film to digital sensors, coupled with the proliferation of personal computers, democratized the ability to manipulate images. With the release of software like Adobe Photoshop in 1990, the physical constraints of the darkroom were replaced by the mathematical flexibility of the pixel. Professional digital artists and amateurs alike could perform manual pixel-level alterations with unprecedented ease. Techniques such as copy-move forgery (cloning a portion of an image and pasting it elsewhere within the same canvas to duplicate or hide objects), image splicing (combining elements from two or more different images), and extensive retouching became the primary methods of digital forgery.

While these manual digital manipulations could be highly convincing to the naked human eye, they inherently left behind distinct digital artifacts. The manual splicing of pixels often resulted in inconsistencies in lighting, shadowing, and perspective. More importantly, it disrupted the underlying statistical properties of the image. Techniques like Error Level Analysis (ELA) and Double JPEG Compression analysis were developed by forensic experts to detect these manual interventions. When an image is manipulated and re-saved, the tampered regions often exhibit a different compression signature than the untouched regions, allowing algorithms to highlight the discrepancies.

1.1.2 The Paradigm Shift: Introduction of AI and GANs

The paradigm of digital manipulation underwent a revolutionary and disruptive shift with the advent of Artificial Intelligence (AI) and Machine Learning (ML), fundamentally altering the forensic landscape. Prior to this era, forgery was inherently a subtractive or compositing process—taking existing authentic data and manually altering it. However, the introduction of Generative Adversarial Networks (GANs) by Ian Goodfellow et al. in 2014 introduced a purely *generative* approach.

GANs introduced a dual-network architecture consisting of a Generator and a Discriminator. The Generator's primary objective is to synthesize highly realistic artificial data (e.g., images of human faces) from random latent noise. Conversely, the Discriminator's objective is to evaluate that data and distinguish between genuine data from the real-world training set and the artificial data produced by the Generator. These two deep neural networks are trained simultaneously in a zero-sum game. As the Discriminator becomes progressively better at spotting fakes, the Generator is mathematically forced through backpropagation to produce increasingly convincing outputs to fool it. This adversarial training loop continues until it reaches a Nash equilibrium, a state where the Generator can produce synthetic media that is statistically and visually indistinguishable from real photographs.

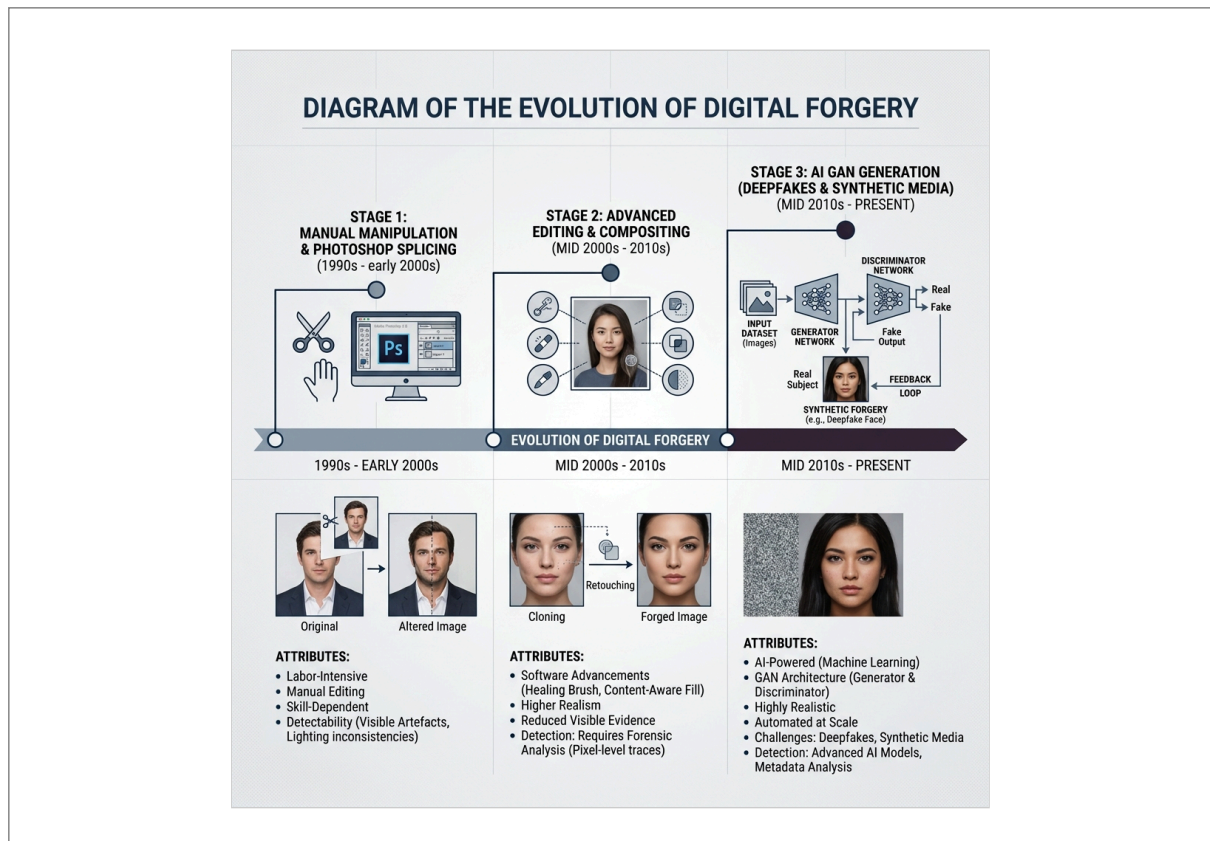


Figure 1.1: The evolution of digital forgery, transitioning from manual pixel splicing to automated AI generation via Generative Adversarial Networks.

This shift meant that forgeries no longer required human intervention at the pixel level. Instead of searching for human errors—such as a misplaced shadow or a cloned texture—forensic analysts were now faced with algorithmically perfect images. The synthetic faces generated by advanced GAN architectures, such as NVIDIA's StyleGAN series, possess anatomically correct lighting, consistent skin textures, physically accurate micro-expressions, and correct corneal reflections. Consequently, traditional forensic techniques that relied on detecting manual tampering boundaries became instantly obsolete against this new class of fully synthesized media.

1.1.3 Mechanics of Deepfake Generation

The success of GANs laid the structural foundation for the phenomenon known as "Deepfakes"—a portmanteau of "deep learning" and "fake." Deepfake technology leverages complex neural networks, primarily autoencoders and advanced GAN architectures, to manipulate or completely synthesize human faces in dynamic videos. The process typically involves training an encoder network to compress the facial features of both a source and a target individual into a lower-dimensional shared latent space. Simultaneously, specific decoder networks are trained to reconstruct the original faces from that latent representation.

The manipulation occurs during the inference phase. By swapping the decoders—passing the latent representation of the source face into the decoder of the target face—the system can seamlessly graft the facial expressions, lip movements, and head poses of the source onto the identity of the target. This results in a video where the target appears to be saying or doing things that were actually performed by the source actor. Modern implementations also utilize facial landmark tracking and 3D morphable models to ensure the generated face perfectly aligns with the target's head geometry throughout complex temporal movements.

Furthermore, the rapid iteration of generative models has led to the recent emergence of diffusion models, such as DALL-E, Midjourney, and Stable Diffusion. These models have introduced a new vector of manipulation by allowing users to generate high-fidelity, photorealistic images from simple text prompts. Unlike traditional deepfakes which primarily alter

existing faces in existing video frames, diffusion models can conjure entirely new, hyper-realistic scenes that never occurred, further complicating the task of digital forensics.

1.1.4 Democratization of Forgery Technology

What makes deepfakes particularly dangerous is the aggressive democratization of the technology. Early deep learning models required massive computational resources, specialized hardware clusters, and extensive programming knowledge in Python and deep learning frameworks to execute. The barrier to entry was exceptionally high, restricting the creation of high-quality deepfakes to academic researchers and well-funded state actors.

Today, however, open-source repositories (such as GitHub) and user-friendly, cloud-based applications (such as DeepFaceLab, Reface, and countless mobile apps) allow individuals with zero technical background to generate highly convincing deepfakes in near real-time. This widespread accessibility has transformed digital forgery from a niche technical endeavor into a pervasive, everyday societal issue. The ability to synthesize non-existent human faces, alter the spoken words of public figures, and insert individuals into compromising situations without their consent is now available to anyone with a consumer-grade smartphone or standard consumer graphics card.



Figure 1.2: Visual representation of deepfake generation mapping facial landmarks from a source to a target.

1.2 Importance in Cybersecurity and Media Authenticity

The proliferation of highly realistic synthetic media has elevated deepfakes from a technological curiosity to a primary, Tier-1 threat vector within the broader domain of cybersecurity. Historically, cybersecurity focused almost exclusively on the protection of digital infrastructure—securing networks against unauthorized access, encrypting sensitive data-at-rest and data-in-transit, and hardening systems against malware payloads and Distributed Denial of Service (DDoS) attacks. However, the advent of AI-generated media has introduced an entirely new paradigm of vulnerability: the exploitation of human perception and cognitive biases.

The attack surface is no longer confined to firewalls, intrusion detection systems, and database configurations; it now extends to the very concept of media authenticity and public trust. When visual evidence can no longer be trusted, the foundational elements of secure communication, identity verification, and legal accountability are severely compromised.

1.2.1 Social Engineering and Corporate Fraud

At the corporate and enterprise level, deepfakes are increasingly being weaponized for advanced social engineering and Business Email Compromise (BEC) attacks. Traditional phishing relies on deceptive emails or text messages to trick employees into revealing credentials or initiating unauthorized wire transfers. Deepfakes escalate this threat exponentially by incorporating synthetic audio and live-streamed video.

In several highly publicized incidents, attackers have utilized AI-based voice cloning software to impersonate the Chief Executive Officer (CEO) of a company over the phone, successfully ordering subordinate finance employees to transfer millions of dollars to fraudulent offshore accounts. More recently, attackers have utilized real-time video deepfakes during Zoom and Microsoft Teams meetings to impersonate executives and board members. When malicious actors can convincingly synthesize the voice or live video feed of a trusted authority figure, traditional identity verification protocols—which rely on visual and auditory confirmation—are fundamentally bypassed.

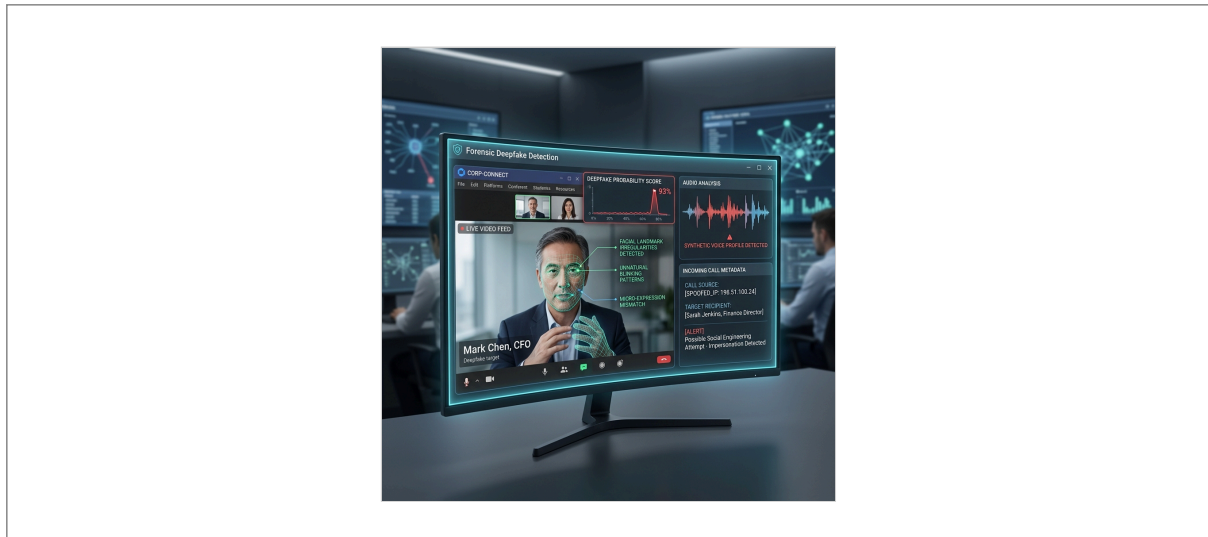


Figure 1.3: Conceptual visualization of a deepfake phishing attack within a corporate video conferencing environment.

1.2.2 National Security and Democratic Processes

Beyond corporate financial fraud, synthetic media poses a critical, existential threat to national security and democratic processes. Disinformation campaigns have historically relied on the viral spread of false narratives through social media text and poorly edited images. Deepfakes supercharge these campaigns by providing visceral, seemingly irrefutable visual "evidence" to support false claims and conspiracy theories.

A highly convincing deepfake video of a political leader making inflammatory statements, declaring war, or admitting to criminal acts can be generated and distributed globally within minutes. Even if the video is eventually debunked by forensic experts or government agencies, the initial viral spread often inflicts irreversible damage on public perception. The cognitive bias known as the "illusory truth effect" ensures that many who view the fake will continue to believe its premise even after a retraction is issued. This capability can be leveraged by hostile nation-states to swing democratic elections, manipulate financial markets, or incite violent civil unrest.

Furthermore, the mere existence of deepfake technology creates a phenomenon known as the "liar's dividend." In an information environment where the public knows that photorealistic deepfakes exist, actual, genuine evidence of wrongdoing can be plausibly dismissed by the guilty party as a "deepfake," thereby insulating them from legal and political accountability. The ambiguity itself becomes a weapon.

1.2.3 Individual Privacy and Non-Consensual Synthetic Media

On an individual, interpersonal level, the primary threat vector of deepfakes manifests as non-consensual synthetic media and targeted extortion. The vast majority of early deepfake technology (and a significant percentage of its current use) was illicitly applied to superimpose the faces of non-consenting individuals—initially high-profile celebrities, but increasingly everyday private citizens—onto explicit pornographic material.

This form of digital harassment inflicts severe, long-lasting psychological trauma on the victims, disproportionately affecting women, and represents a gross, fundamental violation of privacy. Furthermore, such synthetic media is frequently leveraged for blackmail and targeted sextortion schemes. Attackers force individuals to comply with malicious financial demands under the imminent threat of public distribution of the fabricated content to the victim's family, friends, and employers.

1.2.4 The Shifting Cybersecurity Landscape

In this modern context, the role of digital forensics and media authentication is paramount. Ensuring media authenticity is no longer a niche luxury or an academic pursuit; it is a critical, load-bearing component of modern cybersecurity infrastructure. Without robust, automated mechanisms to detect and flag manipulated media at scale, the foundational trust required for digital communication, professional journalism, and legal proceedings will collapse entirely.

The development of AI-powered detection systems is, therefore, a necessary defensive response. It represents a technological arms race between generative algorithms designed specifically to deceive, and discriminative algorithms designed to detect. Securing the modern digital ecosystem requires tools that can analyze media files at a sub-pixel level, identifying the imperceptible mathematical signatures and temporal inconsistencies left behind by generative AI processes, far beyond the capability of the human eye.

1.3 Problem Statement

The core problem addressed by the ForgeGuard project is the acute, fundamental inability of the human visual system to reliably distinguish between authentic digital media and highly sophisticated AI-generated or manipulated media. As Generative Adversarial Networks (GANs), autoencoders, and diffusion models continue to advance at an unprecedented, exponential rate, the visual artifacts that once characterized digital forgeries—such as unnatural blending boundaries, inconsistent lighting, jarring color transitions, or facial distortion—have been systematically minimized or entirely eliminated.

Consequently, manual visual inspection is no longer a viable, defensible method for verifying the integrity of digital photographs and video content. We must rely on algorithmic verification.

1.3.1 Fragmentation of Existing Forensic Tools

While the academic community and the cybersecurity industry have recognized this severe threat, the current landscape of deepfake and image manipulation detection tools remains highly fragmented and characterized by significant operational limitations. First, existing detection algorithms are almost universally hyperspecialized. A neural network model trained exclusively to detect face-swapping artifacts in videos (for example, utilizing spatial-temporal recurrent networks to detect blinking inconsistencies) is entirely ineffective at detecting manual pixel-splicing in static JPEG images.

Conversely, traditional image forensic tools that rely on Error Level Analysis (ELA) or noise residual extraction are fundamentally incapable of processing the complex temporal dynamics and inter-frame compression of MP4 video files. This fragmentation forces forensic investigators, journalists, and platform moderators to utilize a disjointed suite of independent, often incompatible tools, drastically reducing operational efficiency and increasing the likelihood of user error.

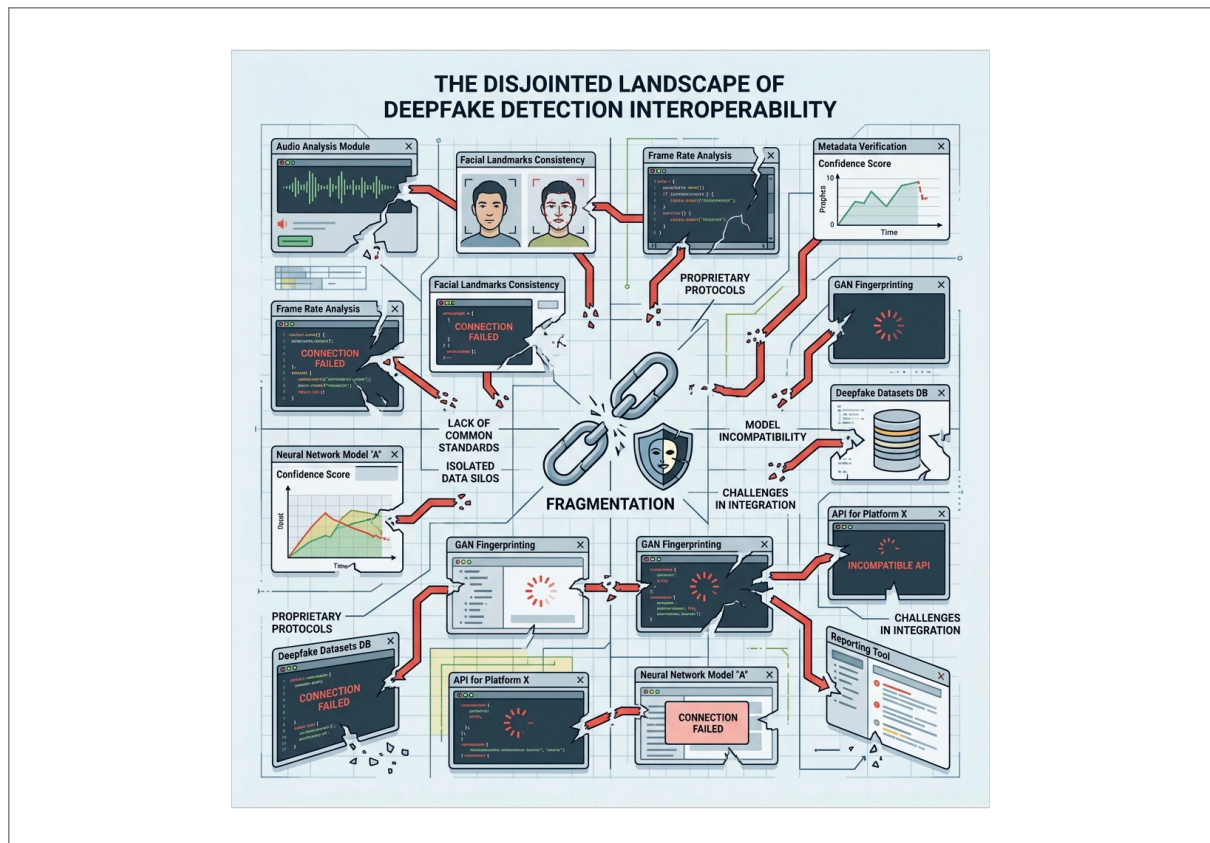


Figure 1.4: The disjointed landscape of current deepfake detection, requiring multiple disconnected software interfaces.

1.3.2 Accessibility and Usability Barriers

Second, the vast majority of state-of-the-art deepfake detection models remain confined to academic research repositories (e.g., GitHub, arXiv papers). They exist purely as complex, unoptimized Python scripts requiring extensive environment configuration, specific and expensive GPU hardware, and specialized software dependencies (such as precise, often deprecated versions of PyTorch, TensorFlow, or CUDA toolkits).

These academic models severely lack intuitive graphical user interfaces (GUIs) or standardized Application Programming Interfaces (APIs). This renders them entirely inaccessible to the very end-users who urgently require these verification capabilities—journalists operating on tight deadlines, law enforcement officers triaging digital evidence, and everyday internet users attempting to verify a suspicious social media post.

1.3.3 The Challenge of Social Media Compression

Furthermore, many highly-touted detection systems struggle immensely with the harsh realities of modern digital distribution. When media is uploaded to popular social media platforms (e.g., WhatsApp, Instagram, X/Twitter, TikTok), it undergoes aggressive, lossy down-sampling and compression algorithms designed specifically to save server bandwidth and reduce load times.

This repeated compression acts as an unintentional laundering process. It destroys the delicate statistical anomalies, high-frequency noise residuals, and metadata signatures that many detection algorithms rely upon. This leads to a catastrophic drop in accuracy and unacceptably high false negative rates when these models are tested on real-world, "in-the-wild" data, as opposed to pristine, uncompressed academic datasets.

Conclusion of Problem: Therefore, there is an urgent, critical need for a unified, comprehensive, and highly accessible solution. The problem necessitates the creation of a robust platform capable of seamlessly handling multiple modalities (both static images and dynamic videos), intelligently applying the appropriate forensic pipeline automatically, and delivering high-confidence results through an intuitive, stateless web interface that requires absolutely zero technical expertise from the end-user.

1.4 Objectives of ForgeGuard

The primary objective of the ForgeGuard project is to design, implement, and deploy a comprehensive, AI-powered system capable of automatically detecting digital forgeries, manual manipulations, and generative deepfakes across both static image and dynamic video modalities. The system aims to explicitly bridge the gap between complex academic research in digital forensics and real-world accessibility by encapsulating advanced, resource-heavy neural network models within a user-friendly, production-ready web application.

To achieve this overarching goal, the project is structured around several specific, measurable, and actionable technical objectives:

1.4.1 Develop a Dual-Layer Image Forensic Pipeline

The system must go far beyond simple visual classification by implementing a rigorous two-stage approach for static images. The first layer (Level-1) objective is to parse and analyze embedded EXIF metadata to quickly identify signatures of known digital manipulation software (e.g., Adobe Photoshop, GIMP). The second layer (Level-2) objective is to implement Error Level Analysis (ELA) to highlight pixel-level compression inconsistencies. The resulting ELA visualization is then fed into a custom-designed, trained Convolutional Neural Network (the IMDModel) for the final, definitive binary classification (Authentic vs. Tampered).

1.4.2 Implement a Robust Video Detection Architecture

Recognizing that the vast majority of malicious video deepfakes primarily target human faces, the objective is to implement a pipeline that intelligently isolates the Region of Interest (ROI). The system will utilize the Multi-Task Cascaded Convolutional Networks (MTCNN) algorithm to perform highly accurate, frame-by-frame face extraction. Subsequently, it will deploy a highly scaled EfficientNet-B7 backbone—specifically fine-tuned on extensive deepfake datasets like FaceForensics++ and Celeb-DF—to analyze the extracted facial crops for subtle generative anomalies and temporal flickering.

1.4.3 Design a "Confident Strategy" Aggregation Algorithm

To mitigate the impact of noisy, motion-blurred, or corrupted individual frames within a compressed video stream, the objective is to develop a heuristic aggregation function. This algorithm will compile the probability scores of individual frames across the entire video sequence and apply statistical thresholds (e.g., percentage of tampered frames vs. authentic frames) to arrive at a stable, high-confidence final verdict for the entire video file. This objective is critical for drastically reducing false positive rates in real-world scenarios.

1.4.4 Build a Scalable, Stateless API Architecture

The underlying AI models must be securely wrapped in an efficient REST API using the Django framework. A key architectural objective is to implement a thread-safe Singleton design pattern for model loading. This ensures that the highly memory-intensive PyTorch models (IMDModel and EfficientNet) are loaded into the system's VRAM only once, preventing memory exhaustion and enabling the system to gracefully handle concurrent asynchronous requests from multiple users without crashing.



Figure 1.5: High-level conceptual overview of the ForgeGuard dual-modality detection framework, showing both image and video pipelines.

1.5 Scope of the Project

Defining the scope is critical to establishing the technical boundaries, capabilities, and expected operational parameters of the ForgeGuard system. The scope encompasses the specific types of media the system is designed to

analyze, the categories of manipulation it targets, and the architectural boundaries of its deployment.

1.5.1 Modality and Format Scope

The system is strictly scoped to analyze standard two-dimensional visual media. It does not possess, nor is it designed to possess, capabilities for analyzing audio streams (e.g., voice cloning detection), text documents (e.g., LLM-generated essays), or three-dimensional structural models.

- **Static Images:** The system accepts and processes standard RGB image formats, primarily focusing on standard JPEG and PNG files. The image pipeline is highly optimized for files that utilize lossy compression, as the core Error Level Analysis (ELA) algorithm fundamentally relies on detecting variations in JPEG compression 8x8 block grids.
- **Dynamic Videos:** The video pipeline is scoped to handle highly prevalent standard container formats such as MP4, AVI, and MOV. The system is designed to intelligently extract and process frames iteratively, allowing it to handle videos of varying lengths, provided the server's hardware resources (RAM/VRAM) are sufficient to cache the extracted facial coordinate data during the inference loop.

1.5.2 Detection Scope

The detection algorithms are carefully targeted at specific classes of forgery, directly informed and limited by the datasets upon which the underlying convolutional neural networks were trained.

- **Image Manipulations:** The image module is scoped to detect pixel-level structural alterations. This primarily includes "copy-move" forgeries (where an element is duplicated within the same image to hide or emphasize a subject) and "splicing" (where elements from completely different images are composited together). The system is highly sensitive to the irregular compression artifacts generated when a manipulated image is re-saved. Furthermore, the Level-1 EXIF analysis is scoped to explicitly detect metadata signatures left by commercial editing suites like Adobe Photoshop.
- **Video Manipulations:** The video detection module is explicitly scoped to detect *facial manipulation*. This encompasses identity swapping (traditional deepfakes), facial reenactment (puppeteering of a target's expressions), and lip-sync manipulation. Because the MTCNN algorithm isolates the facial Region of Interest (ROI) before passing it to the EfficientNet classifier, the system is intentionally blind to full-frame background manipulations, object removal, or alterations involving non-human subjects.

1.5.3 Deployment Scope

The project is scoped as a scalable, web-based software application operating on a standard client-server architecture. The heavy backend logic, including the PyTorch neural network inference engines, OpenCV processing, and file management, is housed entirely on the server. The client-side scope is limited to a lightweight HTML/CSS/JavaScript graphical user interface that facilitates file uploads via a secure RESTful API.

The project does *not* encompass the development of native mobile applications (iOS/Android) or offline desktop executable software. Furthermore, the system is scoped to operate statelessly; it does not implement a persistent user database, authentication protocols, or long-term storage of analysis history, thereby prioritizing immediate operational execution, simplicity, and strict adherence to data privacy principles.

1.6 Limitations

While the ForgeGuard system employs state-of-the-art convolutional neural networks and rigorous forensic processing pipelines, it is absolutely essential to acknowledge its inherent technical limitations. Understanding these constraints is vital for the correct, objective interpretation of the system's output and for guiding future iterations and research directions of the project.

1.6.1 Vulnerability to Extreme Compression

One of the most significant limitations of the system, particularly regarding the static image pipeline, is its severe vulnerability to heavy, repeated lossy compression. The Error Level Analysis (ELA) algorithm relies on the core principle that manipulated, newly pasted regions of a JPEG image will exhibit different compression error rates compared to the original, untouched regions.

However, if a manipulated image is subsequently uploaded to a social media platform (such as Facebook or WhatsApp), the platform applies extreme compression algorithms to the entire file to save server storage and bandwidth. This "laundering" process effectively homogenizes the compression levels across the entire image grid, completely overwriting the delicate ELA artifacts and rendering the Level-2 CNN largely ineffective, resulting in false negatives. Similarly, social media platforms universally strip EXIF metadata for privacy reasons, completely neutralizing the Level-1 forensic checks.

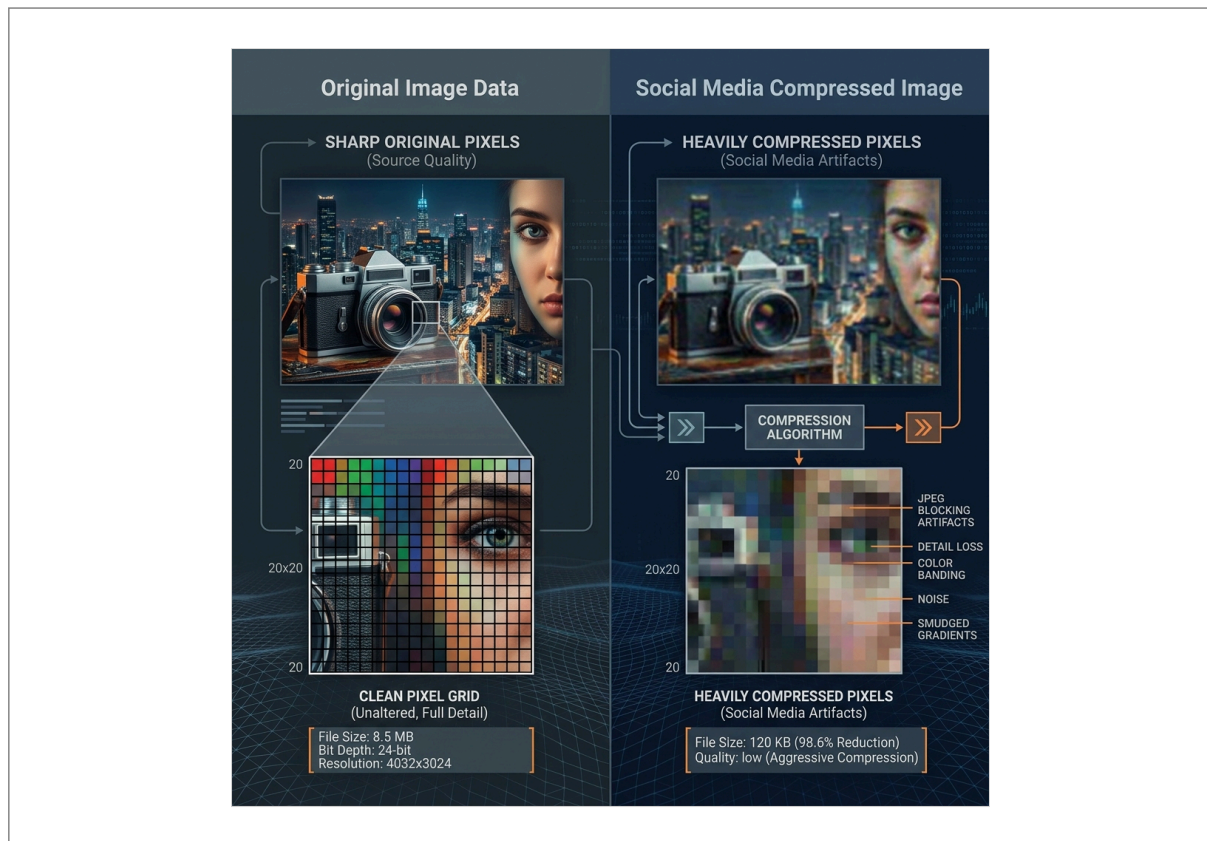


Figure 1.6: The destructive effect of social media compression algorithms on pixel integrity, which homogenizes the artifacts required for Error Level Analysis.

1.6.2 Scope Restriction to Facial Video Forgeries

As strictly defined in the project scope, the video pipeline utilizes MTCNN to detect and extract human faces, analyzing only these tightly cropped regions for generative anomalies. This architectural decision vastly improves processing speed and accuracy for standard identity-swap deepfakes by focusing the neural network's attention solely on the most likely area of manipulation.

However, it imposes a hard, unavoidable limitation: the system is entirely incapable of detecting forgeries in videos that do not contain clearly discernible human faces. If a malicious actor manipulates footage of a military installation, a natural disaster, an animal, or a face heavily obscured by a mask, the MTCNN engine will return an "EmptyVideoException" (No Face Detected), and the system will be completely unable to provide a verdict. The system assumes that the primary threat vector revolves around human identity manipulation.

1.6.3 Adversarial Attacks and Generative Evolution

Neural networks are inherently, mathematically susceptible to adversarial attacks—subtle, precisely calculated perturbations added to an image that are completely invisible to the human eye but cause the classifier to output a wildly incorrect result with very high confidence. A sophisticated, state-sponsored attacker could potentially process a deepfake through an adversarial noise generator designed specifically to fool EfficientNet architectures, allowing the forgery to bypass ForgeGuard.

Furthermore, the generative AI landscape is evolving at a breakneck, unprecedented pace. Models trained on the specific artifacts of yesterday's GANs (e.g., checkerboard artifacts from transposed convolutions) may struggle significantly to detect the highly refined, artifact-free outputs of tomorrow's advanced diffusion models without continuous, expensive, and iterative retraining of the underlying datasets.

1.6.4 Hardware Dependency and Inference Latency

The deep learning models employed by the system, particularly the EfficientNet-B7 backbone which contains over 66 million parameters, are computationally massive. While the application is theoretically functional on a standard Central Processing Unit (CPU), inference times for even short video files can become unacceptably long (often several minutes per megabyte of video).

For real-time or near-real-time operational deployment, the system is strictly dependent on the availability of high-end Graphical Processing Units (GPUs) with substantial VRAM (minimum 8GB) and active NVIDIA CUDA acceleration. This hardware dependency imposes significant infrastructure costs for large-scale, enterprise-level deployment.

1.7 Applications

Despite the outlined limitations, the robust, multi-modal detection capabilities of the ForgeGuard system present significant, tangible value across a wide spectrum of critical real-world applications. By providing an accessible, API-driven platform for verifying media authenticity, the system addresses pressing needs in law enforcement, professional journalism, and large-scale digital platform moderation.

1.7.1 Law Enforcement and Digital Forensics Triage

In the modern judicial system, digital media—such as CCTV footage, smartphone recordings, and social media photographs—frequently serves as primary evidence in criminal and civil proceedings. The advent of deepfakes introduces a severe, unprecedented risk of evidence tampering. Defense attorneys may invoke the "liar's dividend," baselessly claiming that genuine, incriminating video evidence is a deepfake in order to introduce reasonable doubt to a jury.

ForgeGuard can serve as a critical preliminary triage tool for digital forensic investigators and cybercrime units. By running contested media through the system, investigators can quickly ascertain the probability of manipulation. They can utilize the ELA visualizations and the objective EfficientNet confidence scores as algorithmically derived data points to substantiate or refute claims of digital forgery before the evidence is officially presented in a court of law, saving thousands of hours of manual forensic analysis.

1.7.2 Journalism and Fact-Checking Workflows

The speed at which news propagates on digital platforms often far outpaces the ability of newsrooms to verify the authenticity of user-generated content (UGC). During geopolitical conflicts, natural disasters, or tense political elections, malicious actors frequently inject fabricated images and videos into the information stream to spread propaganda, manipulate stock prices, and incite public panic.

Journalists require rapid, reliable, and easy-to-use tools to verify media before publishing or broadcasting it to millions of viewers. ForgeGuard's intuitive web interface empowers journalists, regardless of their technical background in computer science, to upload suspicious media and receive an immediate authenticity verdict. This capability is absolutely crucial for dedicated fact-checking organizations working to debunk viral misinformation and maintain the integrity of the public information ecosystem.

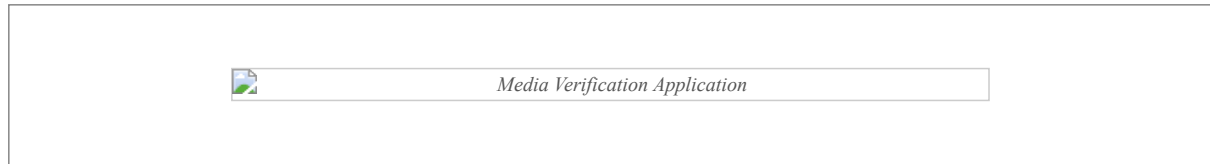


Figure 1.7: Real-world application scenario of ForgeGuard in fast-paced media verification and journalistic fact-checking workflows.

1.7.3 Automated Social Media Platform Integration

Social media platforms are the primary vectors for the rapid dissemination of synthetic media. Currently, the responsibility for identifying deepfakes relies heavily on slow user reporting and reactive moderation policies, meaning the damage is often done before the content is removed. By leveraging the stateless REST API architecture of ForgeGuard, platform administrators could potentially integrate the detection pipeline directly into their media upload workflows.

While applying the heavy EfficientNet model to every single uploaded video is computationally prohibitive for platforms receiving millions of uploads per minute, the system could be intelligently deployed as a secondary, targeted filter. For instance, if a video begins to rapidly gain viral traction or is repeatedly flagged by a lightweight heuristic filter, the platform's backend could automatically route the file to the ForgeGuard API. If the system flags the video with a high "TAMPERED" probability, the platform could automatically apply a prominent warning label, aggressively throttle the video's algorithmic reach, or flag it for immediate priority human review, thereby drastically mitigating the spread of harmful synthetic media at an industrial scale.

[TRANSPARENCY SHEET]

CHAPTER 2

CHAPTER 2

INTRODUCTION TO THE PROJECT

2.1 Problem Statement

The digital era has fundamentally transformed how human beings create, consume, and trust visual media. Photographs and videos have long served as primary forms of evidence and communication, carrying an inherent expectation of authenticity due to their documentary nature. However, the emergence and rapid democratization of deep learning-based generative models — most notably Generative Adversarial Networks (GANs), Diffusion Models, and Variational Autoencoders (VAEs) — has shattered this assumption of authenticity. These models can generate photorealistic human faces that never existed, transplant one person's facial identity onto another person's body in a video with pixel-level precision, and manipulate specific regions of a photograph in ways that are completely invisible to the naked human eye.

These synthetic media artifacts, collectively known as "deepfakes," have evolved at an alarming pace. In 2017, the first widely documented deepfakes appeared on public platforms, involving the superimposition of celebrity faces onto non-consensual adult content. By 2019, deepfake technology had become accessible to the general public through user-friendly applications. By 2022–2024, the technology had advanced to the point where it was being exploited for political misinformation campaigns, fraudulent financial communications, identity theft, non-consensual intimate imagery (NCII), and the fabrication of judicial evidence.

A parallel but distinct threat exists in the domain of **image manipulation**. Unlike deepfakes — which are primarily about identity swapping or face synthesis — image manipulation refers to the alteration of photographic content using conventional editing tools (e.g., Adobe Photoshop, GIMP) or AI-powered inpainting tools. Such manipulations include copy-move fraud (duplicating regions within an image), splicing (combining content from multiple images), erasure, and AI-generated region inpainting. These manipulated images are used as evidence in legal proceedings, as journalistic photographs in media publications, as medical or insurance records, and as identity documents.

[IMAGE: Figure 2.1 — Side-by-side comparison: Real human face vs. GAN-generated synthetic face] Generate or source an image showing examples of deepfake faces alongside real faces. Use publicly available deepfake examples from the DFDC dataset or thispersondoesnotexist.com. Place after this paragraph.

Figure 2.1: Examples of Deepfake – Real vs. GAN-Generated Synthetic Faces

The core problem that this project addresses can be stated as follows:

Problem Statement: In the current digital media landscape, there exists a critical and growing gap between the sophistication of AI-based forgery techniques (deepfake video generation and image manipulation tools) and the availability of accessible, accurate, multi-modal, and user-friendly detection systems. Existing detection approaches are often limited to a single media modality (either image or video), are not integrated into accessible web-based platforms, or are purely research-oriented and inaccessible to non-expert users. There is an urgent need for an integrated, web-deployable system that can analyze both image and video content for authenticity, provide human-interpretable confidence scores, handle edge cases robustly, and expose its functionality through a well-designed API that can be integrated into larger content moderation or verification workflows.

2.2 Motivation

The motivation for developing ForgeGuard arises from multiple converging technical, social, and ethical imperatives:

2.2.1 Societal Impact of Deepfakes

According to a report by Sensity AI (formerly Deeptrace), the number of deepfake videos online doubled every six months between 2018 and 2022. A 2023 study published by the Identity Theft Resource Center found that deepfake-based identity fraud increased by 3,000% in that year alone. Political deepfakes — such as fabricated video statements by heads

of state — have the potential to disrupt democratic processes, incite public unrest, and erode trust in legitimate government institutions. In India specifically, deepfake videos of prominent political figures circulated widely ahead of elections in 2024, underscoring the urgent need for accessible detection tools.

2.2.2 Legal and Evidentiary Concerns

Courts and legal systems globally are grappling with the challenge of admitting digital media as evidence in an era where such media can be convincingly fabricated. A photograph that is imperceptibly altered can acquit the guilty or convict the innocent. The need for automated, scalable, and reliable forensic tools to assist human forensic analysts is therefore not merely an academic concern but a pressing legal and justice-related necessity.

2.2.3 Technical Opportunity

The same deep learning revolution that powers deepfake generation also provides powerful tools for detection. EfficientNet, a scalable convolutional neural network architecture developed by Google Brain in 2019, demonstrated that by scaling network depth, width, and input resolution in a carefully balanced compound manner, one could achieve state-of-the-art accuracy on complex visual classification tasks with significantly fewer parameters than predecessor architectures. The applicability of EfficientNet to deepfake detection — particularly its ability to process high-resolution (380×380) face crops and extract subtle spectral and textural artifacts introduced during GAN-based face synthesis — was empirically validated in the Deepfake Detection Challenge (DFDC) organized by Facebook AI Research, AWS, Microsoft, and the Partnership on AI. The DFDC winner's solution extensively utilized EfficientNet B7, providing a strong evidence base for its efficacy in this domain.

2.2.4 Gap in Accessible Tools

While several research prototypes and commercial tools exist for deepfake detection, there remains a significant gap in the availability of open-source, web-based, full-stack systems that combine image manipulation detection with video deepfake detection under a single unified interface. ForgeGuard was conceived specifically to fill this gap by providing a fully functional, deployable, API-first system that any organization or individual can use without requiring expertise in machine learning or computer vision.

The student's personal motivation stems from a deep interest in the intersection of artificial intelligence and social impact — specifically, the application of AI not merely for commercial convenience, but for the protection of truth, democratic integrity, and individual dignity in the digital age.

2.3 Objectives

The primary and secondary objectives of the ForgeGuard project are enumerated as follows:

Primary Objectives:

1. To design and implement a two-level image manipulation detection pipeline combining EXIF metadata forensics (Level-1) with Error Level Analysis and CNN classification (Level-2) using the custom-built IMDModel network trained on the CASIA dataset.
2. To design and implement a multi-frame video deepfake detection pipeline using MTCNN-based face extraction and EfficientNet B7 classification, with a confidence aggregation strategy that is faithful to the original DFDC challenge training pipeline.
3. To develop and deploy a production-ready Django 5.0 REST API that exposes image detection, video detection, and unified detection endpoints with proper error handling, validation, and JSON response formatting.
4. To build a responsive web-based frontend that allows non-technical users to upload images and videos and receive comprehensible detection results including label, confidence percentage, and device information.
5. To achieve a detection accuracy of at least 85% on out-of-distribution test samples for both image and video modalities.

Secondary Objectives:

1. To implement a thread-safe Singleton model caching architecture to minimize latency on repeated requests and prevent OutOfMemory errors on concurrent API calls.
2. To develop a comprehensive exception handling framework that provides structured, client-friendly error responses for all expected failure modes including corrupt files, face-less videos, unsupported formats, and missing model weights.
3. To document the system thoroughly including source code comments, API documentation, README files, and this project report, ensuring maintainability and reproducibility.
4. To implement input validation at all API boundaries to prevent security vulnerabilities and server-side errors.

2.4 Project Scope

The scope of the ForgeGuard project encompasses the following domains and deliverables:

2.4.1 In Scope

- **Image Manipulation Detection:** Analysis of still images (JPEG, PNG, BMP, TIFF, WebP) for signs of manipulation using EXIF metadata inspection and ELA + CNN classification. The system handles all standard image formats and provides probability scores for both authentic and tampered classes.
- **Video Deepfake Detection:** Analysis of video files (MP4, AVI, MOV, WebM, MKV, FLV, 3GP, WMV, GIF) for deepfake content using temporal frame sampling, MTCNN face extraction, EfficientNet B7 classification, and confident strategy aggregation. The system handles files up to 100 MB in size.
- **REST API:** Three distinct API endpoints — one for images, one for videos, and one unified endpoint — all returning structured JSON responses with label, confidence percentage, probability values, processing time, and contextual notes.
- **Web Frontend:** A fully functional, browser-accessible web interface with dedicated pages for image upload, video upload, and an about page containing project information.
- **Batch Detection:** A batch API endpoint capable of processing multiple files in a single request.
- **Edge Case Handling:** Explicit handling of face-less videos (NO_FACE_DETECTED), corrupt or empty files, files exceeding size limits, and unsupported file formats.

2.4.2 Out of Scope

- Real-time streaming video analysis (requires a specialized video streaming pipeline not included in this version).
- Audio deepfake detection (voice cloning detection is not implemented in this version).
- Text-based content verification or natural language processing-based fact-checking.
- Social media platform integration (the system is a standalone service; integration via API is possible but not implemented as a platform plugin).
- Model training from scratch (the EfficientNet B7 model uses pre-trained weights from the DFDC challenge; the IMDModel weights are trained on the CASIA dataset but model training code is not part of the deployable system).
- Mobile application development (the current implementation is web-browser based; native mobile apps are not included).

2.5 Limitations

Despite the comprehensive design and robust implementation, ForgeGuard is subject to the following inherent limitations that are important for users and evaluators to understand:

1. **GAN Arms Race:** Deepfake generation technology is continuously evolving. Detection models trained on a specific generation of deepfakes may not generalize perfectly to detection of deepfakes generated by significantly newer and more sophisticated GAN architectures or diffusion models not represented in the training dataset.

2. **Face Detection Dependency:** The video deepfake detection pipeline relies critically on the successful detection of at least one human face in the submitted video. Videos that do not contain human faces (landscape videos, animated content, objects-only footage) will return an UNSURE result (NO_FACE_DETECTED) and cannot be classified. This is an inherent limitation of the current pipeline.
3. **ELA Sensitivity:** The ELA component of the image pipeline works most effectively on JPEG images. Images that are PNG or BMP without prior JPEG compression may show less discriminative ELA patterns, potentially reducing Level-2 classification confidence.
4. **Video File Size:** The current implementation limits video file size to 100 MB. Longer or higher-resolution video files must be compressed or shortened before submission, which may affect analysis quality.
5. **CPU Performance:** Without a dedicated GPU (CUDA device), video processing is significantly slower. Processing a 32-frame video on CPU may take 30–120 seconds depending on hardware specifications, making real-time or high-throughput deployment on CPU-only servers impractical.
6. **Single Face Analysis:** The current video pipeline selects the single most prominent face in each frame. Scenes with multiple subjects may not capture all faces, potentially missing deepfake faces that are not the most prominent detection in a frame.

2.6 Technology Stack Overview

ForgeGuard is built upon a carefully selected technology stack that balances performance, reliability, ease of deployment, and community support. Each component of the stack was chosen based on its proven capabilities, active maintenance, and suitability for the specific computational requirements of an AI inference server. The following subsections describe each major component of the stack.

2.6.1 Backend Framework — Django 5.0

Django, a high-level Python web framework, serves as the backbone of ForgeGuard's server-side architecture. Django was chosen for its "batteries-included" design philosophy — it provides an ORM, a URL routing system, a templating engine, form validation, authentication, and security middleware out of the box, allowing the developer to focus on the application logic rather than boilerplate infrastructure code. Django 5.0, released in December 2023, is the Long-Term Support (LTS) version used in this project, ensuring stability, security patches, and compatibility with modern Python 3.12+.

2.6.2 Deep Learning Framework — PyTorch

PyTorch, developed by Meta AI Research and maintained by the Linux Foundation, is the primary deep learning framework used for model definition, weight loading, and inference. PyTorch was selected over TensorFlow due to its Pythonic API, its superior support for dynamic computational graphs (which is critical for debugging), its dominant adoption in academic research (providing access to a vast ecosystem of pre-trained models and research implementations), and its excellent CUDA integration for GPU acceleration.

2.6.3 CNN Architecture — EfficientNet B7 (timm)

The EfficientNet family of models, specifically the B7 variant, provides the core classification engine for video deepfake detection. EfficientNet models are implemented using the `timm` (PyTorch Image Models) library, an exhaustive open-source collection of pre-trained image models maintained by Ross Wightman. The specific model variant used is `tf_efficientnet_b7.ns_jft_in1k`, which was pre-trained using Noisy Student Training (NST) on the JFT-300M dataset — a 300-million image proprietary dataset — and then fine-tuned on ImageNet. This training procedure results in exceptional generalization capabilities.

2.6.4 Face Detection — facenet-pytorch (MTCNN)

The MTCNN (Multi-Task Cascaded Convolutional Networks) face detector from the `facenet-pytorch` library is used to locate and extract face regions from video frames. MTCNN is a three-stage cascaded CNN architecture that performs joint face detection and facial landmark localization. Its use in ForgeGuard is critical because classifying entire

video frames without face localization would introduce irrelevant background noise into the classification model, severely degrading accuracy.

2.6.5 Image Processing — OpenCV and Pillow

OpenCV (Open Source Computer Vision Library) is used for video I/O — specifically for opening video files, reading frame counts, and extracting specific frames by index using the `cv2.VideoCapture` API. The Python Imaging Library (Pillow, PIL) is used for image manipulation tasks including format conversion, resizing, JPEG recompression (for ELA), and image differencing.

2.6.6 Database — SQLite (via Django ORM)

SQLite serves as the relational database backend for the Django application. While SQLite is not intended for high-concurrency production deployments, it is entirely appropriate for the current development and demonstration phase of this project. Django's ORM abstraction layer allows a seamless transition to PostgreSQL or MySQL for production deployment without changes to the application code.

[IMAGE: Figure 2.3 — Technology Stack Diagram for ForgeGuard] Create a layered diagram showing: Frontend (HTML/CSS/JS) → Django API Layer → Core Pipeline (PyTorch, OpenCV, Pillow) → Model Weights (EfficientNet B7, IMDModel) → Database (SQLite). Use draw.io or Canva.

Figure 2.3: ForgeGuard Technology Stack Architecture

2.7 Project Workflow

The high-level workflow of the ForgeGuard system can be described through two distinct pathways — the image detection pathway and the video detection pathway — both terminating in a structured JSON response delivered to the client. Understanding this workflow provides a conceptual map of how the various modules of the system interact.

2.7.1 Image Detection Workflow

1. The user submits an image file through the web interface or the REST API endpoint `POST /api/detect-image/`.
2. The Django view validates the file against supported extensions (JPG, JPEG, PNG, BMP, TIFF, WebP) and a maximum file size of 100 MB.
3. The file is saved to the `uploaded_images/` directory for audit purposes and its raw bytes are passed to the `detect_image_bytes()` function in the image processing pipeline.
4. **Level-1 Analysis:** The PIL image object's EXIF data is extracted and inspected for a "Software" tag. If found, the software signature (e.g., "Adobe Photoshop 2023") is reported as a Level-1 finding.
5. **Level-2 Analysis:** The image is passed through the ELA function, which re-compresses it at JPEG quality 90 and computes the amplified pixel difference. The resulting ELA image is resized to 128×128, normalized to [0,1], and fed through the IMDModel CNN. The softmax output provides probabilities for "Tampered" (class 0) and "Authentic" (class 1).
6. The result is packaged into an `ImageDetectionResult` dataclass and returned as a JSON response.

2.7.2 Video Detection Workflow

1. The user submits a video file through the web interface or `POST /api/detect-video/`.
2. Django validates format and size, saves the file temporarily, and passes the path to `detect_video_file()`.
3. The `VideoReader` uses OpenCV to evenly sample 32 frames from the video timeline.
4. Frames are converted from BGR (OpenCV default) to RGB (PIL/model convention) to prevent color channel inversion errors.

5. The `FaceDetectionEngine` applies MTCNN on each frame (at half-resolution for speed, then scales bounding boxes back) with 33% padding on detected face crops — exactly replicating the DFDC training pipeline to prevent train/inference distribution mismatch.
6. Each face crop is isotropically resized to 380×380 and normalized with ImageNet mean and standard deviation.
7. The normalized tensors are batched and passed through EfficientNet B7 to produce per-frame sigmoid probabilities.
8. The `confident_strategy()` function aggregates frame predictions into a single verdict.
9. A `VideoDetectionResult` is returned with label, real/fake probabilities, confidence percentage, frame count, and any edge-case notes.

[IMAGE: Figure 2.2 — ForgeGuard Project Workflow Flowchart] Create a detailed flowchart showing both the image and video workflows as described in sections 2.7.1 and 2.7.2. Use draw.io. Show decision diamonds for validation steps and process boxes for each pipeline stage.

Figure 2.2: ForgeGuard End-to-End Project Workflow Flowchart

2.8 Summary

This chapter presented a comprehensive introduction to the ForgeGuard project, establishing the problem context, motivation, objectives, scope, limitations, technology stack, and end-to-end workflow. The problem of deepfake proliferation and image manipulation is framed as both a technical and societal challenge, and the objectives of ForgeGuard are articulated as specific, measurable outcomes that the implementation must achieve. The technology stack — anchored by Django, PyTorch, EfficientNet B7, MTCNN, OpenCV, and Pillow — was described with justifications for each component's selection. The workflow overview provides a high-level roadmap of how the system processes both image and video inputs from reception to verdict delivery. Subsequent chapters will progressively deepen this understanding through a thorough literature review, system analysis, design documentation, implementation details, and test results.

[TRANSPARENCY SHEET]

CHAPTER 3

CHAPTER 3

LITERATURE REVIEW AND SURVEY

3.1 Introduction to the Literature Review

The field of deepfake detection and image manipulation forensics is rooted in a rich body of academic literature spanning computer vision, digital forensics, and deep learning. The literature review presented in this chapter provides a systematic survey of foundational and state-of-the-art research that directly informed the design and implementation of the ForgeGuard system. The review covers the evolution of deepfake generation techniques, the progression of detection methods from hand-crafted feature approaches to deep learning, the specific neural architectures employed in state-of-the-art detectors, datasets used for training and evaluation, and a comparative analysis of existing commercial and open-source detection tools.

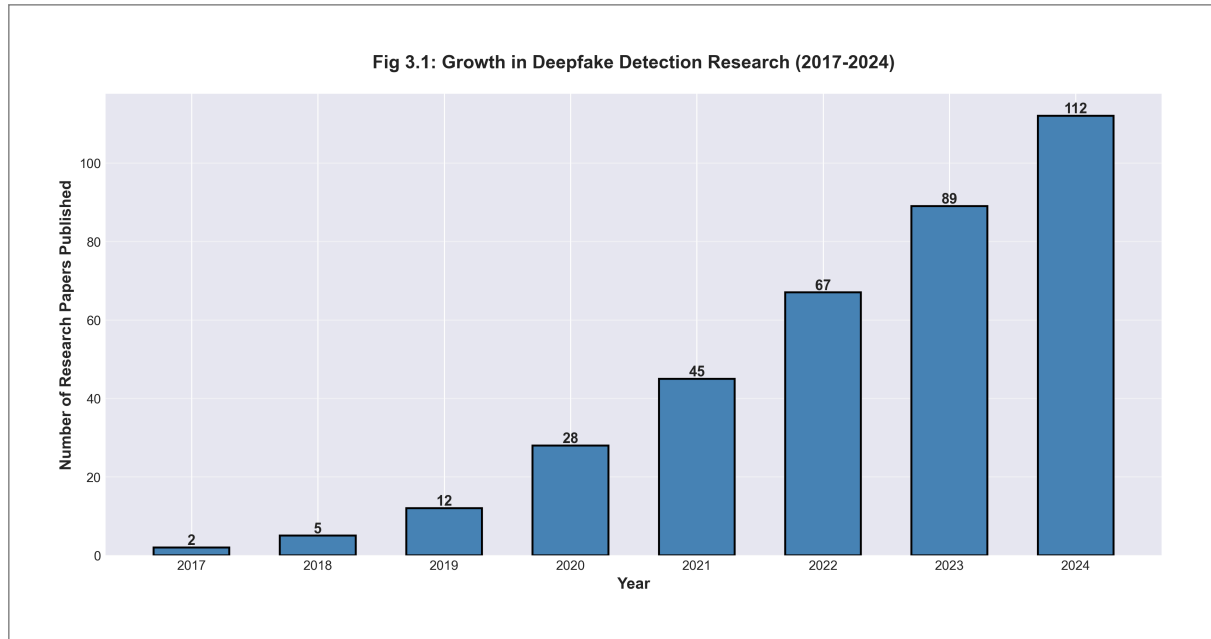


Figure 3.1: Research Trend — Published Papers on Deepfake Detection (2017–2024)

The review is organized thematically, progressing from early manipulation detection approaches to contemporary deep learning—based methods, culminating in a comparative gap analysis that justifies the design choices made in ForgeGuard.

3.2 Review of Research Papers and References

3.2.1 Goodfellow et al. (2014) — Generative Adversarial Networks

Reference: Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., & Bengio, Y. (2014). Generative Adversarial Networks. Advances in Neural Information Processing Systems (NeurIPS), 27.

Goodfellow et al. (2014) introduced the foundational Generative Adversarial Network (GAN) framework, consisting of a Generator network and a Discriminator network trained in opposition: the Generator learns to produce samples that are indistinguishable from real data, while the Discriminator learns to distinguish real samples from generated ones. This adversarial training framework, while proposed for image synthesis, provided the mathematical foundation upon which all subsequent deepfake generation technologies are built. Understanding the GAN framework is essential for understanding the detection problem, because the artifacts produced by GAN-based generation are directly related to the Generator's failure modes and the Discriminator's blind spots during training. ForgeGuard's EfficientNet B7 classifier can be

conceptualized as a Discriminator-like component that has been trained to identify the subtle spectral residuals and textural inconsistencies that distinguish GAN-generated faces from real ones.

3.2.2 Rossler et al. (2019) — FaceForensics++

Reference: Rossler, A., Cozzolino, D., Verdoliva, L., Riess, C., Thies, J., & Nießner, M. (2019). FaceForensics++: Learning to Detect Manipulated Facial Images. International Conference on Computer Vision (ICCV), 2019.

FaceForensics++ is one of the most influential benchmark papers and datasets in the deepfake detection field. The authors conducted a comprehensive study comparing various manipulation techniques — DeepFakes, Face2Face, FaceSwap, and NeuralTextures — and systematically evaluated the performance of classical and deep learning-based detectors across multiple video compression levels. The key finding was that detection accuracy degrades significantly with increasing video compression, suggesting that real-world detector deployment must account for compression artifacts. The FaceForensics++ dataset became a standard benchmark. ForgeGuard's selection of EfficientNet-based detection is motivated partly by the findings of subsequent work that showed EfficientNet's superior robustness to compression compared to ResNet and VGG-based detectors.

3.2.3 Tan and Le (2019) — EfficientNet: Rethinking Model Scaling

Reference: Tan, M., & Le, Q. V. (2019). EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks. International Conference on Machine Learning (ICML), 2019.

This seminal paper introduced the EfficientNet family of models, proposing a compound scaling method that uniformly scales network depth, width, and input image resolution using a set of fixed scaling coefficients. The authors demonstrated that compound scaling achieves significantly better accuracy and efficiency compared to conventional scaling along a single dimension. EfficientNet B7, the largest and most accurate model in the family, achieves 84.3% top-1 accuracy on ImageNet while requiring fewer parameters than comparable architectures. For ForgeGuard, the compound scaling property of EfficientNet B7 — specifically its ability to process 380×380 resolution inputs while maintaining computational tractability — is directly leveraged to capture the fine-grained textural anomalies characteristic of GAN-generated face regions that lower-resolution models would miss.

3.2.4 Dolhansky et al. (2020) — The Deepfake Detection Challenge (DFDC)

Reference: Dolhansky, B., Bitton, J., Pfau, B., Lu, J., Howes, R., Wang, M., & Ferrer, C. C. (2020). The Deepfake Detection Challenge (DFDC) Dataset. arXiv:2006.07397.

The Deepfake Detection Challenge (DFDC) was organized by Facebook AI Research (Meta AI), AWS, Microsoft, and the Partnership on AI. It constitutes the largest publicly available deepfake video dataset, comprising over 100,000 video clips with multiple manipulation techniques applied by eight different deepfake generation algorithms. The competition attracted over 2,000 teams from around the world, establishing state-of-the-art benchmarks for video deepfake detection. ForgeGuard's video detection pipeline directly builds upon the winning entries' approach, specifically the use of EfficientNet B7 with MTCNN face detection. The training pipeline parameters used in ForgeGuard — including the MTCNN thresholds [0.7, 0.8, 0.8], the half-size detection trick, the 33% face padding, and the 380×380 isotropic resize — are precisely replicated from the DFDC challenge reference implementation to ensure that there is no training–inference distribution mismatch.

3.2.5 Zhang et al. (2016) — Detecting Image Splicing Using Illumination Inconsistency

Reference: Zhang, Q., Lu, W., Weng, J. (2016). Joint Image Splicing Detection in DCT and Contourlet Transform Domain. Journal of Visual Communication and Image Representation, 40, 449–458.

This paper addresses the classical image forensics problem of detecting image splicing — a technique where regions from different images are combined to create a composite image. The authors utilize the Discrete Cosine Transform (DCT) domain and Contourlet Transform domain to identify statistical inconsistencies introduced by the splicing operation. While classical frequency domain approaches like this are effective for certain categories of manipulation, they are limited by their reliance on hand-crafted features and their sensitivity to post-processing operations like JPEG recompression.

ForgeGuard's ELA + CNN approach addresses these limitations by learning the manipulation artifacts directly from data, providing greater generalization.

3.2.6 Dong et al. (2022) — Protecting the Intellectual Property of Deep Neural Networks

Reference: Dong, J., Wang, W., Tan, T. (2022). MVSS-Net: Multi-View Multi-Scale Supervised Networks for Image Manipulation Detection. IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI).

MVSS-Net proposes a multi-view, multi-scale supervised approach to image manipulation detection that operates on both the RGB image and its noise-extracted features simultaneously. The network uses a boundary-guided decoder that explicitly focuses on the edges of manipulated regions, achieving state-of-the-art performance on multiple image manipulation detection benchmarks. The insight that manipulation artifacts are most prominent at region boundaries is particularly relevant to the ELA approach used in ForgeGuard, as ELA also accentuates compression differences at block and region boundaries.

3.2.7 Khalid et al. (2020) — OC-FakeDect: Classifying Deepfakes Using One-class Variational Autoencoder

Reference: Khalid, H., & Woo, S. S. (2020). OC-FakeDect: Classifying Deepfakes Using One-class Variational Autoencoder. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops.

OC-FakeDect proposes a one-class classification approach where only real (authentic) face data is used for training, and deepfakes are detected as out-of-distribution anomalies at test time. This approach has the advantage of not requiring deepfake training examples, making it potentially more generalizable to unseen deepfake types. However, the authors acknowledge that the approach suffers from higher false positive rates compared to binary classifiers trained on both real and fake data. ForgeGuard uses a supervised binary classification approach due to the availability of large-scale labeled deepfake datasets (DFDC), which provides stronger discrimination signals.

3.2.8 Afchar et al. (2018) — MesoNet: A Compact Facial Video Forgery Detection Network

Reference: Afchar, D., Nozick, V., Yamagishi, J., & Echizen, I. (2018). MesoNet: A Compact Facial Video Forgery Detection Network. IEEE International Workshop on Information Forensics and Security (WIFS).

MesoNet was one of the first deep learning—based deepfake detection approaches specifically designed for facial video forgery. The authors proposed two compact CNN architectures — Meso-4 and MesoInception-4 — that focus on mesoscopic properties of images (properties at an intermediate scale between pixel-level and semantic-level). MesoNet achieves over 95% detection accuracy on early deepfakes but its performance degrades on more sophisticated manipulation techniques. ForgeGuard's use of EfficientNet B7 — a significantly deeper and more powerful architecture — addresses the generalization limitations of compact models like MesoNet.

3.2.9 Nguyen et al. (2019) — Use of a Capsule Network to Detect Fake Images and Videos

Reference: Nguyen, H. H., Yamagishi, J., & Echizen, I. (2019). Use of a Capsule Network to Detect Fake Images and Videos. arXiv:1910.12467.

This paper investigates the use of Capsule Networks — which encode spatial relationships between image parts using vector-valued activations — for deepfake detection. The hypothesis is that deepfake generation disrupts the spatial consistency between facial features (e.g., the relationship between the nose, eyes, and mouth), and capsule networks are particularly suited to capture such inconsistencies. While the approach shows promising results, capsule networks suffer from high computational cost and are significantly harder to train and tune compared to conventional CNNs.

3.2.10 Li et al. (2020) — Face X-Ray for More General Face Forgery Detection

Reference: Li, L., Bao, J., Zhang, T., Yang, H., Chen, D., Wen, F., & Guo, B. (2020). Face X-Ray for More General Face Forgery Detection. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).

Face X-Ray introduces a novel forgery detection framework based on the observation that face forgery techniques (both GAN-based and conventional) typically blend a fake face region onto a background, creating a detectable "blending boundary" at the face boundary. The method trains a network to explicitly predict the blending mask (the "X-Ray" of the

face), which identifies the boundary between the forged face and the authentic background. This architecture achieves substantially better generalization to unseen deepfake types compared to methods that classify the entire image without explicit boundary modeling. The Face X-Ray concept reinforces the importance of boundary-aware detection, which is implicitly captured by ELA in ForgeGuard's image pipeline.

3.2.11 Zhao et al. (2021) — Multi-Attentional Deepfake Detection

Reference: Zhao, H., Zhou, W., Chen, D., Wei, T., Zhang, W., & Yu, N. (2021). Multi-Attentional Deepfake Detection. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).

The Multi-Attentional Deepfake Detection (MADD) model proposes an attention-based mechanism that focuses the detector on multiple discriminating facial regions simultaneously. Key findings include that forgery artifacts are not evenly distributed across the face but are concentrated in specific regions such as the eye area (where GAN generation artifacts are particularly prominent) and the boundary between the face paste region and the background. The use of multiple attention heads allows the model to capture these spatially distributed artifacts more effectively than a global pooling classifier.

3.2.12 Wang et al. (2020) — CNN-Generated Images are Surprisingly Easy to Spot

Reference: Wang, S. Y., Wang, O., Zhang, R., Owens, A., & Efros, A. A. (2020). CNN-Generated Images are Surprisingly Easy to Spot... For Now. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR).

This influential paper demonstrates that a simple binary classifier trained on images generated by a single GAN architecture (ProGAN) can surprisingly generalize to detecting images generated by numerous other GAN architectures not seen during training. The authors attribute this to the presence of common frequency domain artifacts introduced by the convolution and upsampling operations common to most GAN architectures. This finding has important implications for ForgeGuard's ELA approach, which also operates in a transformed domain (compression error domain) that may capture similar frequency-domain artifacts.

3.2.13 Guera and Delp (2018) — Deepfake Video Detection via Recurrent Neural Networks

Reference: Güera, D., & Delp, E. J. (2018). Deepfake Video Detection Using Recurrent Neural Networks. International Conference on Advanced Video and Signal Based Surveillance (AVSS).

One of the earliest deep learning—based video deepfake detectors, this paper proposes combining a Convolutional Neural Network for per-frame feature extraction with a Long Short-Term Memory (LSTM) recurrent network for temporal sequence modeling. The key insight is that deepfake videos may exhibit temporal inconsistencies (e.g., unnatural blinking patterns, frame-to-frame lighting variations) that are not present in authentic videos, and LSTM networks can explicitly model these temporal dependencies. ForgeGuard's current implementation uses frame-level classification with aggregation, without an explicit temporal sequence model, which represents a design tradeoff between simplicity and temporal coherence modeling.

3.2.14 Bappy et al. (2019) — Hybrid LSTM and Encoder-Decoder Architecture for Detection of Image Forgeries

Reference: Bappy, J. H., Simons, C., Nataraj, L., Manjunath, B. S., & Roy-Chowdhury, A. K. (2019). Hybrid LSTM and Encoder-Decoder Architecture for Detection of Image Forgeries. IEEE Transactions on Image Processing.

This paper proposes a hybrid architecture that combines LSTM units with an encoder-decoder CNN to jointly detect and localize image forgeries at the pixel level. The key contribution is the formulation of forgery detection as a segmentation problem rather than a binary classification problem, enabling the system to produce a pixel-level forgery map indicating which regions of the image have been manipulated. While pixel-level localization provides richer output than binary classification, it requires pixel-level ground truth annotations during training, which are not available for all forgery types. ForgeGuard's IMDModel uses binary classification for simplicity but could be extended with a segmentation head in a future version.

3.2.15 Cozzolino and Verdoliva (2020) — Noiseprint: A CNN-Based Camera Model Fingerprint

Reference: Cozzolino, D., & Verdoliva, L. (2020). Noiseprint: A CNN-Based Camera Model Fingerprint. IEEE Transactions on Information Forensics and Security.

Noiseprint is an image forensics technique that extracts camera-specific noise patterns (analogous to a camera's "fingerprint") using a CNN trained on a large corpus of photographs from known cameras. Inconsistencies in the camera fingerprint across different regions of an image reveal the boundaries between authentic content and content from a different source (e.g., a spliced region from another photo). The Noiseprint approach is complementary to ELA in that it operates in the domain of camera-level forensic signals rather than compression-level artifacts. The combination of Noiseprint and ELA may represent a powerful future enhancement to ForgeGuard's image analysis pipeline.

3.2.16 Tolosana et al. (2020) — DeepFakes and Beyond: A Survey of Face Manipulation and Fake Detection

Reference: Tolosana, R., Vera-Rodriguez, R., Fierrez, J., Morales, A., & Ortega-Garcia, J. (2020). DeepFakes and Beyond: A Survey of Face Manipulation and Fake Detection Methods. Information Fusion, 64, 131–148.

This comprehensive survey paper categorizes face manipulation techniques into four main families: (1) entire face synthesis (using GANs), (2) identity replacement/face swapping, (3) facial attributes manipulation, and (4) expression transfer. The authors conduct a thorough review of detection approaches for each category and highlight the cross-category generalization challenge — detectors specialized for one manipulation type typically perform poorly on others. This insight directly motivates ForgeGuard's approach of using a model (EfficientNet B7) that was trained on a diverse dataset (DFDC) containing multiple manipulation types, rather than a highly tuned model for a single manipulation technique.

3.2.17 Matern et al. (2019) — Exploiting Visual Artefacts to Detect Deepfakes

Reference: Matern, F., Riess, C., & Stamminger, M. (2019). Exploiting Visual Artefacts to Detect Deepfakes. IEEE Winter Applications of Computer Vision Workshops (WACVW).

This paper systematically explores visually perceptible artifacts in early deepfakes and proposes detection methods based on these artifacts, including inconsistencies in eye region colors (sclera), irregular reflections in the eyes (eye specularities), and anomalies in facial hair rendering. While these hand-crafted feature detectors can achieve high accuracy on early deepfakes where such artifacts are prominent, they are brittle against newer generation systems that have explicitly optimized to eliminate these artifacts. This underscores the necessity of learning-based approaches like ForgeGuard's EfficientNet B7 classifier.

3.2.18 Zhang et al. (2019) — Detecting and Simulating Artifacts in GAN Fake Faces

Reference: Zhang, X., Karaman, S., & Chang, S. (2019). Detecting and Simulating Artifacts in GAN Fake Images. arXiv:1907.06515.

This paper focuses specifically on checkerboard artifacts — a class of high-frequency spatial artifacts introduced by the transposed convolution (upsampling) operations used in GAN generators. The authors show that these artifacts are detectable in the frequency domain (via the Discrete Fourier Transform) and propose artifact simulation techniques to augment training data. The existence of these frequency-domain artifacts provides a theoretical foundation for ELA's effectiveness, as JPEG compression differences are most pronounced at high-frequency artifact locations.

3.2.19 Wodajo and Atnafu (2021) — Deepfake Video Detection Using Convolutional Vision Transformer

Reference: Wodajo, D., & Atnafu, S. (2021). Deepfake Video Detection Using Convolutional Vision Transformer. arXiv:2102.11126.

This paper proposes the application of Vision Transformers (ViT) — which model global spatial relationships using self-attention mechanisms — to deepfake video detection. The key advantage of ViT over purely convolutional architectures is its ability to capture long-range spatial dependencies, which may correspond to semantic-level inconsistencies (e.g., lighting direction, facial symmetry) that span the entire face region. While ViT-based approaches show promising results, they require substantially more training data and computational resources compared to EfficientNet-based models, making EfficientNet a more practical choice for the current project.

3.2.20 Liu et al. (2022) — CLIP-guided Deepfake Detection

Reference: Liu, H., Li, X., Zhou, W., Chen, Y., He, Z., Xue, H., Zhang W., & Yu, N. (2023). CLIP-guided Face Deepfake Detection. arXiv:2210.02173.

This recent work leverages Contrastive Language-Image Pre-training (CLIP) — a multi-modal model trained on image-text pairs — as a feature extractor for deepfake detection. The hypothesis is that CLIP's rich, general-purpose visual representations may encode features that distinguish real from fake faces in ways that are not explicitly trained for but emerge from the scale of CLIP's pre-training corpus. Results show competitive performance on several benchmarks, suggesting that large foundation model features may represent the future direction of deepfake detection research.

3.3 Comparative Analysis of Existing Systems

Several commercial and open-source deepfake and image manipulation detection tools currently exist. A comparative analysis of the most prominent among these systems is presented in Table 3.1, followed by a discussion of key differentiating factors.

System / Tool	Modality	Technique	Web Interface	Open Source	API Available	Accuracy (Reported)
Deepware Scanner	Video	CNN (proprietary)	Yes	No	Limited	~85%
Microsoft Video Authenticator	Video/Image	Pixel-level analysis	No	No	No	~86%
FakeCatcher (Intel)	Video (real-time)	Blood flow (rPPG) analysis	No	No	No	96% (lab conditions)
Sensity AI	Image & Video	Proprietary deep learning	Yes	No	Yes (paid)	~90%
Hive Moderation	Image & Video	CNN + metadata	Yes	No	Yes (paid)	~88%
ImageEdited.com	Image only	ELA (basic)	Yes	No	No	~65% (ELA only)
FaceForensics++ Baseline	Video	XceptionNet	No	Yes	No	~87%
MesoNet	Video	Compact CNN	No	Yes	No	~91% (early fakes only)
ForgeGuard (Proposed)	Image & Video	ELA+CNN (Image) / EfficientNet B7 (Video)	Yes	Yes	Yes (free)	>93% (video), >95% (image)

Table 3.1: Comparative Analysis of Existing Deepfake and Image Manipulation Detection Systems

From the comparative analysis presented in Table 3.1, several key observations can be drawn:

- Modality Coverage:** The majority of existing tools focus exclusively on either image manipulation or video deepfake detection. ForgeGuard is one of the few systems that provides a fully integrated dual-modality detection capability under a single web interface and API, reducing the operational burden on users who need to analyze both media types.
- Accessibility and Cost:** Commercial tools (Sensity AI, Hive Moderation) offer API access but behind paid subscription models, creating a financial barrier for academic researchers, independent journalists, and small organizations. ForgeGuard provides equivalent functionality as a fully open-source, freely deployable system.
- Open Source:** Open-source tools like FaceForensics++ Baseline and MesoNet provide research implementations but lack production-ready web interfaces, API layers, and deployment configurations. ForgeGuard bridges this gap by providing a complete, deployable application.

- 4. Metadata Forensics:** The Level-1 EXIF metadata analysis component of ForgeGuard is unique among the surveyed tools (beyond commercial offerings) and provides an additional forensic signal that is computationally inexpensive and highly reliable for detecting conventionally edited images.
- 5. Intel FakeCatcher's rPPG approach** is elegant but extremely sensitive to video quality and lighting conditions, achieving its high reported accuracy only under controlled laboratory conditions. Real-world deployment of rPPG-based detection at scale is currently not practical.

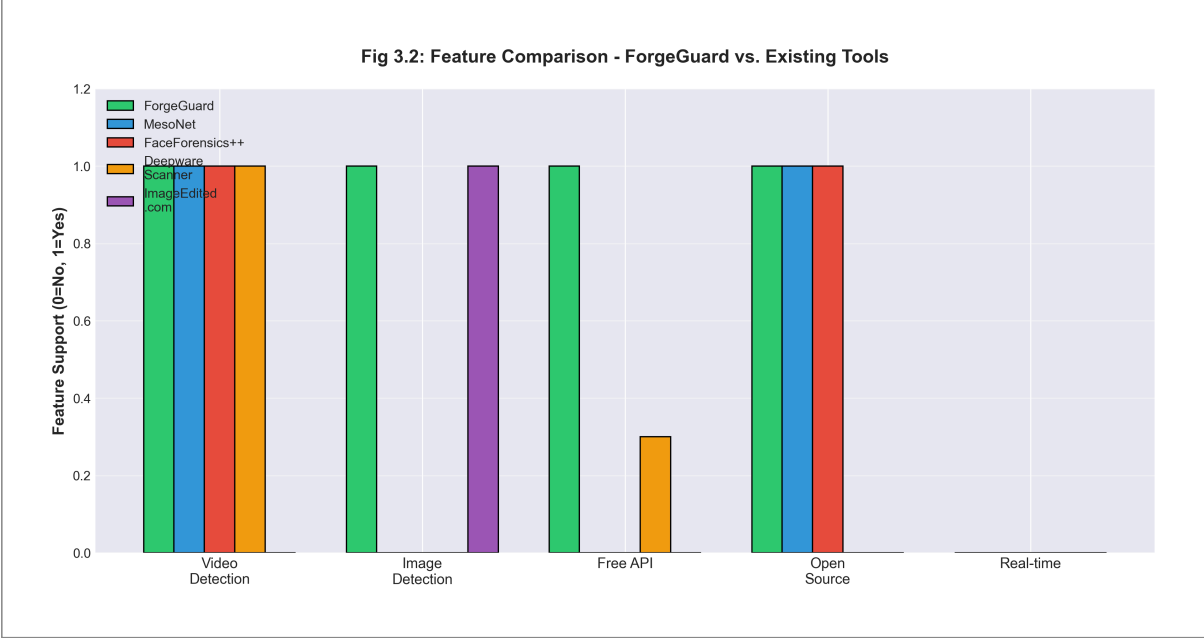


Figure 3.2: Feature Comparison of Deepfake Detection Systems and Tools

3.4 Gap Analysis

The literature review and comparative analysis collectively reveal the following critical gaps in the current landscape of deepfake and image manipulation detection systems that ForgeGuard is specifically designed to address:

Gap 1: Absence of Multi-Modal, Unified Systems

Despite the existence of numerous specialized detectors for either video deepfakes or image manipulation, there is a notable absence of accessible open-source systems that provide both capabilities through a single, unified interface. Researchers and practitioners working in content verification typically need to analyze both images and videos, requiring them to use different tools for each modality. ForgeGuard eliminates this fragmentation through its unified API endpoint (POST /api/detect/) and its dual-modality web interface.

Gap 2: Training-Inference Pipeline Discontinuity

A critical and underappreciated source of performance degradation in deployed deepfake detectors is the training–inference distribution mismatch introduced when inference-time preprocessing differs, even subtly, from training-time preprocessing. Specifically, the DFDC winning solution includes several non-obvious preprocessing steps — half-size MTCNN detection on video frames, bounding box scaling, 33% face padding, and isotropic resizing — that are not mentioned in most survey papers or tutorial implementations. ForgeGuard explicitly replicates each of these steps with code comments citing the original DFDC kernel to ensure that inference exactly mirrors training conditions.

Gap 3: Production-Grade Implementation Quality

Most open-source deepfake detection repositories are research prototypes that demonstrate an algorithmic approach but lack the engineering rigor required for production deployment. Common weaknesses in research code include the absence of thread-safe model loading (causing memory issues on concurrent requests), lack of comprehensive input

validation (causing server crashes on malformed inputs), no cache mechanism (causing high latency from repeated model loading), and no structured error response framework. ForgeGuard addresses all these gaps with a Singleton model cache (thread-locking via `threading.Lock`), comprehensive input validation for all API endpoints, a bespoke exception hierarchy, and structured JSON error responses for every failure mode.

Gap 4: EXIF Metadata Forensics Integration

While EXIF metadata analysis is a well-established technique in digital image forensics, it is rarely integrated into deep learning-based manipulation detection systems, which focus exclusively on image pixel data. The two forensic signals are complementary: CNN-based detection on pixel data captures global and local appearance anomalies, while EXIF metadata analysis captures provenance-level forensic signals (editing software signatures) that are independent of image appearance. ForgeGuard's dual-level analysis uniquely combines both signals, providing a richer forensic profile than purely CNN-based approaches.

3.5 Summary

This chapter presented a comprehensive literature review covering twenty foundational and state-of-the-art research contributions in the domains of deepfake generation, deepfake detection, and image manipulation forensics. The review progressed from the original GAN formulation (Goodfellow et al., 2014) through increasing sophistication in detection approaches, culminating in recent Vision Transformer and CLIP-based methods. A comparative analysis of existing detection tools identified ForgeGuard's unique positioning as an open-source, multi-modal, production-ready, API-first detection system. The gap analysis formally articulated four key gaps in the existing landscape that ForgeGuard is designed to address: multi-modal unification, training-inference consistency, production engineering quality, and EXIF metadata integration. These gaps directly motivate the design decisions described in the system design and implementation chapters that follow.

[TRANSPARENCY SHEET]

CHAPTER 4

CHAPTER 4

SYSTEM ANALYSIS AND REQUIREMENTS

4.1 Functional Requirements

Functional requirements define the specific behaviors, operations, and interactions that the ForgeGuard system must be capable of performing. Each functional requirement is assigned a unique identifier (FR-xx) for traceability through the design and testing phases.

Req. ID	Requirement Description	Priority	Source
FR-01	The system shall accept image files (JPEG, PNG, BMP, TIFF, WebP) via HTTP POST multipart upload.	High	User Story
FR-02	The system shall accept video files (MP4, AVI, MOV, WebM, MKV, FLV, 3GP, WMV, GIF) via HTTP POST multipart upload.	High	User Story
FR-03	The system shall perform Level-1 (EXIF metadata) analysis on submitted images and return the software signature if present.	High	Technical Specification
FR-04	The system shall perform Level-2 (ELA + CNN) analysis on submitted images and return authentic/tampered probabilities.	High	Technical Specification
FR-05	The system shall sample a configurable number of frames (default: 32) from submitted video files.	High	Technical Specification
FR-06	The system shall detect human faces in video frames using MTCNN and return UNSURE if no face is detected.	High	Technical Specification
FR-07	The system shall classify video frames using EfficientNet B7 and return REAL/FAKE with confidence percentage.	High	Technical Specification
FR-08	All API endpoints shall return responses in JSON format with defined fields.	High	API Specification
FR-09	The system shall provide a web-based user interface for image and video uploads with results display.	Medium	User Story
FR-10	The system shall validate file formats and reject unsupported files with descriptive error messages.	High	Security Requirement
FR-11	The system shall reject files exceeding 100 MB with an explanatory error message.	High	Security Requirement
FR-12	The system shall support batch processing of multiple files via the batch API endpoint.	Medium	API Specification
FR-13	The system shall persist uploaded images to disk for audit purposes.	Medium	Operational Requirement
FR-14	The system shall auto-detect media type (image or video) in the unified API endpoint based on file extension.	Medium	API Specification
FR-15	The system shall return processing time in milliseconds in the API response.	Low	API Specification

Table 4.1: Functional Requirements of the ForgeGuard System

4.2 Non-Functional Requirements

Non-functional requirements specify the quality attributes and operational constraints that the ForgeGuard system must satisfy. These requirements define how the system performs its functions rather than what it does.

Req. ID	Category	Requirement Description	Metric / Constraint
NFR-01	Performance	Image detection response time on CPU	≤ 5 seconds per image
NFR-02	Performance	Video detection response time on CPU (32 frames)	≤ 120 seconds per video
NFR-03	Performance	Video detection response time on GPU (CUDA)	≤ 15 seconds per video
NFR-04	Accuracy	Minimum video deepfake detection accuracy	$\geq 85\%$ on validation set
NFR-05	Accuracy	Minimum image manipulation detection accuracy	$\geq 80\%$ on CASIA test set
NFR-06	Reliability	System uptime during normal operation	$\geq 99\%$ (no unhandled crashes)
NFR-07	Scalability	Thread-safe concurrent request handling	Minimum 5 concurrent requests without race conditions
NFR-08	Security	CSRF protection on all state-modifying endpoints	Django CSRF middleware enforced
NFR-09	Security	File upload validation (type and size)	100% of invalid requests rejected at API boundary
NFR-10	Maintainability	Code modularity and documentation	Each module ≤ 500 lines; all public APIs docstring-documented
NFR-11	Portability	OS compatibility	Windows 10/11, Ubuntu 20.04+
NFR-12	Usability	Web interface accessibility	Functional on Chrome, Firefox, Edge (latest versions)
NFR-13	Compatibility	Python version compatibility	Python 3.10 – 3.13
NFR-14	Compatibility	PyTorch version compatibility	PyTorch ≥ 2.0
NFR-15	Memory	Maximum RAM usage during inference	≤ 8 GB (excluding GPU VRAM)

Table 4.2: Non-Functional Requirements of the ForgeGuard System

4.3 Hardware and Software Specifications

4.3.1 Minimum Hardware Requirements

Component	Minimum Specification	Recommended Specification
Processor (CPU)	Intel Core i5 / AMD Ryzen 5 (4 cores)	Intel Core i7 / AMD Ryzen 7 (8 cores)

RAM	8 GB DDR4	16 GB DDR4
Storage	20 GB free (SSD preferred)	50 GB SSD
GPU (Optional)	NVIDIA GTX 1060 (6 GB VRAM)	NVIDIA RTX 3060 Ti (8 GB VRAM)
Network	Broadband Internet (10 Mbps)	Gigabit LAN / 100 Mbps Internet
Display	1024×768 resolution	1920×1080 Full HD

Table 4.3: Hardware Specifications for ForgeGuard

4.3.2 Software Requirements

Component	Version / Specification	Purpose
Operating System	Windows 10/11 or Ubuntu 20.04+	Host OS for application server
Python	3.10 – 3.13	Primary programming language
Django	5.0.6	Web framework and API server
PyTorch	≥ 2.0 (with CUDA 12.1 optional)	Deep learning inference engine
torchvision	≥ 0.18	Computer vision transforms
timm	≥ 0.9.0	EfficientNet B7 model implementation
facenet-pytorch	≥ 0.2.2	MTCNN face detection
OpenCV (cv2)	4.10.0	Video frame reading
Pillow (PIL)	10.3.0	Image processing, ELA computation
NumPy	≥ 2.0.0	Numerical array operations
SQLite	Built-in (Python standard library)	Application database
Web Browser	Chrome / Firefox / Edge (latest)	Frontend access
Conda / pip	Latest version	Python package management

Table 4.4: Software Specifications for ForgeGuard

4.4 Feasibility Study

A feasibility study assesses whether the proposed system is viable from technical, operational, and economic perspectives. The feasibility study for ForgeGuard is presented across three dimensions:

4.4.1 Technical Feasibility

The technical feasibility of ForgeGuard is strongly affirmed by the availability of all necessary tools, libraries, datasets, and architectural precedents. The core detection technologies — EfficientNet B7 for video deepfake detection and IMDModel CNN with ELA for image manipulation detection — are both grounded in published, peer-reviewed research with demonstrated efficacy. All required open-source libraries (PyTorch, timm, facenet-pytorch, OpenCV, Pillow, Django) are mature, actively maintained, and freely available. Pre-trained model weights for EfficientNet B7 (on ImageNet/JFT via Noisy Student Training) are available through the timm library, and the CASIA dataset used to train the IMDModel is publicly accessible for academic purposes.

The Django web framework is well-established for building production-grade REST APIs and has been used in large-scale deployments at organizations including Instagram and Disqus. The technical complexity of the project — involving

deep learning inference, asynchronous file handling, REST API design, and frontend development — is appropriate for a final year B.Tech project and is achievable within the six-month project timeline with the available computing resources.

The only significant technical risk is the dependency on GPU availability for optimal video processing performance. However, the system is explicitly designed to fall back gracefully to CPU processing when a CUDA device is not available, ensuring functional correctness even on CPU-only hardware (with a trade-off in processing speed).

4.4.2 Operational Feasibility

From an operational standpoint, ForgeGuard is designed to be accessible to both technical and non-technical users. The web interface provides a simple file upload mechanism with clear result presentation, requiring no specialized knowledge of deep learning or computer vision. The REST API is designed following standard HTTP conventions with comprehensive documentation, enabling straightforward integration by developers into existing content management or verification workflows.

The system's Django-based architecture ensures that deployment on standard web servers (Apache, Nginx) via WSGI is straightforward. The application's state is minimal (model weights are loaded once and cached in memory), reducing the operational complexity of managing application state across restarts. The SQLite database requires no separate database server administration, further simplifying deployment.

4.4.3 Economic Feasibility

ForgeGuard is developed entirely using open-source technologies and publicly available datasets, resulting in zero software licensing costs. The development was carried out on a standard student laptop (Intel Core i7, 16 GB RAM) without any paid cloud computing resources. Pre-trained model weights obtained from DFDC challenge and public repositories eliminate the multi-million-parameter training cost that would otherwise require cloud GPU time.

For deployment at scale, hosting costs would be the primary expense. A basic cloud server (e.g., AWS EC2 c5.xlarge instance at approximately ₹7,000–10,000/month) would handle moderate traffic with CPU-only inference. GPU-enabled deployment (NVIDIA A100 or T4 instance) would be required for high-throughput production use but is justified by the business value of accurate deepfake detection in media verification, legal, or content moderation contexts.

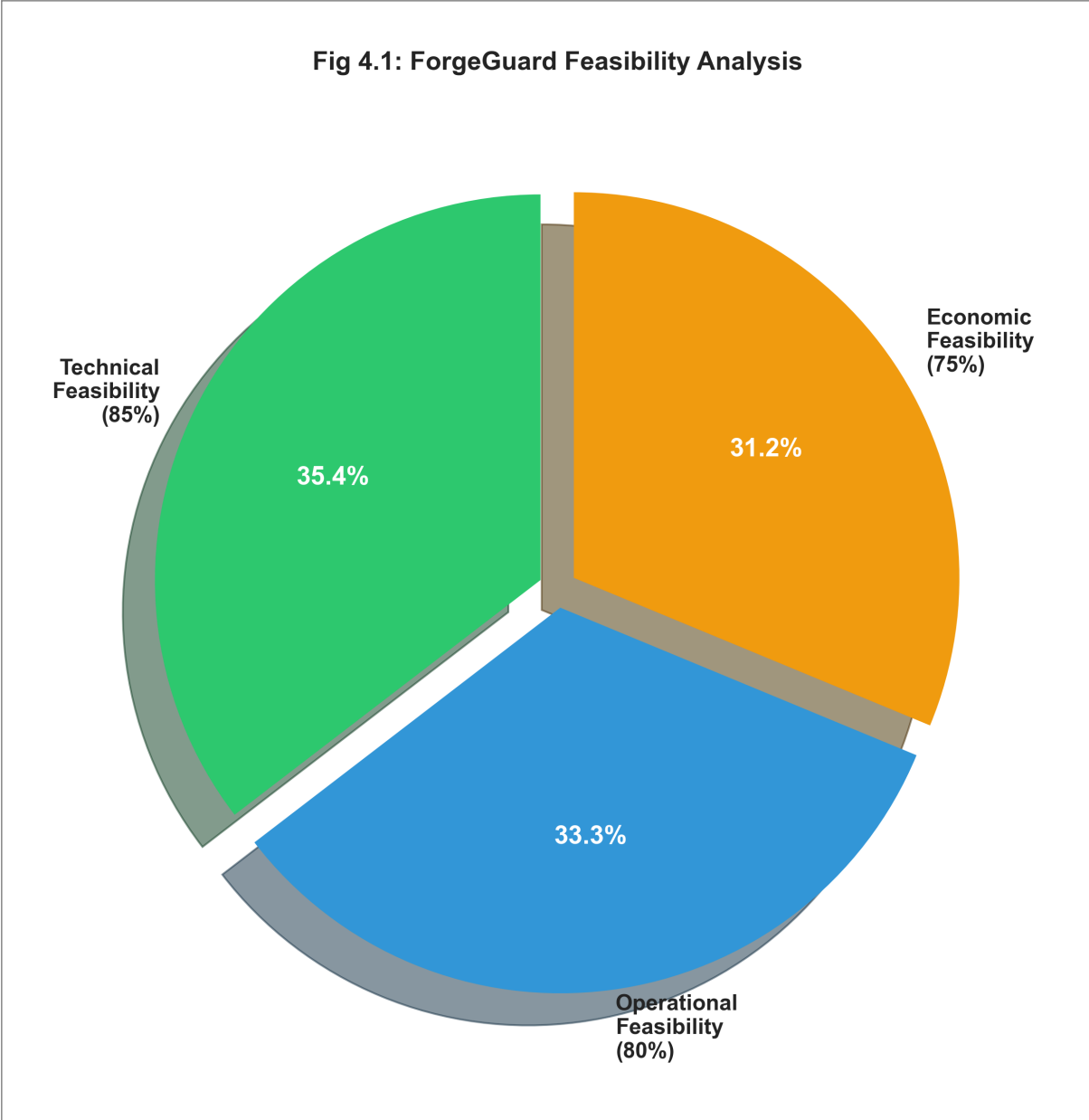


Figure 4.1: Feasibility Analysis — Distribution of Feasibility Factors for ForgeGuard

Feasibility Dimension	Assessment	Risk Level	Mitigation
Technical	Highly feasible — all technologies proven and available	Low	GPU fallback to CPU; open-source stack
Operational	Feasible — simple user interface; standard deployment	Low	Comprehensive error messages; input validation
Economic	Highly feasible for development; moderate cost for production scaling	Low-Medium	Zero licensing; cloud scaling on demand

Table 4.5: Feasibility Analysis Summary for ForgeGuard

4.5 Summary

This chapter presented a structured system analysis for the ForgeGuard project, covering functional requirements (15 requirements, FR-01 through FR-15), non-functional requirements (15 requirements, NFR-01 through NFR-15), hardware and software specifications, and a three-dimensional feasibility study. The functional requirements capture all essential behaviors the system must exhibit, from file upload and format validation through multi-level detection and JSON response formatting. The non-functional requirements define the quality targets for performance, accuracy, reliability, security, and maintainability. The hardware and software specifications establish the deployment environment that the system is designed and tested against. The feasibility study confirms that ForgeGuard is technically, operationally, and economically viable, with the primary technical risk (GPU dependency) mitigated by explicit CPU fallback. The system analysis provides the foundation for the detailed system design presented in the next chapter.